FACULTY OF MEDIA, SCIENCE & TECHNOLOGY

SCHOOL OF COMPUTING & ENGINEERING

DriveSense AI

An Intelligent Automotive Diagnostic and Predictive Maintenance System Using Machine Learning and OBD-II Data

Student Name: Dasheill Vas

Student ID: s5621997

Degree: BSc (Hons) Software Engineering May 2026

Module: Individual Project (40 credits)

Supervisor: **Dr Emili Balaguer-Ballester**

Table of Contents

Declared word count: 4,998 words

Word Count Notice:

The stated word count excludes figure captions, explanatory figure annotations, tables, references, appendices, and supplementary implementation evidence.

1. Abstract

Modern vehicles generate large amounts of diagnostic and sensor data through On-Board Diagnostics (OBD-II) systems; however, this information is often presented as technical fault codes and numerical readings that are difficult for non-technical drivers to interpret. This project addresses this issue through the design and development of DriveSense AI, an intelligent automotive diagnostic and predictive maintenance system that combines OBD-II telemetry, machine learning concepts and conversational artificial intelligence to improve vehicle fault interpretation and maintenance awareness.

The system was developed as a web-based software artefact using Flask, Python, HTML, CSS and JavaScript. Core features include vehicle diagnostics, live telemetry monitoring, AI-assisted fault explanation, predictive maintenance analysis, service history management and diagnostic report generation. Machine learning-based analysis was implemented to estimate vehicle health, repair urgency and safe-to-drive status using diagnostic and sensor data.

The artefact was evaluated through functional testing, scenario-based testing and usability-focused review to assess system reliability, interface usability and diagnostic effectiveness. The results demonstrate that

integrating artificial intelligence with automotive diagnostic systems can improve accessibility, support maintenance decision-making and enhance user understanding of vehicle faults. The project also provides a foundation for future development of intelligent predictive vehicle maintenance platforms.

2. Introduction

Modern vehicles contain increasingly advanced electronic control systems that continuously monitor engine performance, fuel efficiency, braking systems and overall vehicle operation. Most modern vehicles use On-Board Diagnostics II (OBD-II) technology to standardise diagnostic communication and emissions monitoring across manufacturers (SAE International, 2022). These systems generate large amounts of sensor and diagnostic data through Electronic Control Units (ECUs), enabling faults and performance issues to be identified electronically. However, traditional OBD-II diagnostic tools are often designed primarily for professional mechanics and technically experienced users. Many everyday drivers struggle to interpret diagnostic trouble codes, maintenance warnings and repair urgency information, which can lead to delayed maintenance decisions and increased repair costs.Artificial intelligence and machine learning technologies are becoming increasingly important within modern automotive software systems. AI-driven solutions are now used in predictive maintenance, intelligent diagnostics and vehicle monitoring applications. Machine learning techniques are particularly valuable because they can identify patterns within vehicle data and estimate potential failures before severe mechanical issues occur (Carvalho et al., 2019). Predictive maintenance systems can therefore improve safety, reduce repair costs and support proactive vehicle management (Zonta et al., 2020).

Despite these advancements, many existing automotive diagnostic applications still provide limited explanation of detected faults and minimal predictive support for non-technical users. Commercial systems such as Carly, FIXD and BlueDriver offer vehicle scanning and monitoring functionality, but many focus primarily on displaying raw diagnostic information rather than helping users understand the severity and implications of vehicle faults. Research into automotive human-computer interaction has also identified usability limitations in diagnostic systems designed for inexperienced users (Biondi et al., 2019). This demonstrates a gap between the availability of automotive diagnostic data and the ability of ordinary drivers to interpret and use that information effectively.The automotive industry is also becoming increasingly dependent on software and intelligent electronic systems. Modern vehicles may contain over 100 Electronic Control Units and millions of lines of software code responsible for diagnostics, safety and vehicle functionality (Bosch, 2023). In addition, the global predictive maintenance market is expected to exceed $60 billion by 2030 due to growing adoption of artificial intelligence and data-driven maintenance technologies (Fortune Business Insights, 2024). These developments highlight the increasing importance of intelligent diagnostic and predictive maintenance systems within both the automotive and software engineering industries.

This project addresses these challenges through the design and development of DriveSense AI, an intelligent automotive diagnostic and predictive maintenance platform that combines OBD-II telemetry data, machine learning concepts and conversational artificial intelligence within a unified web-based application. The project is positioned within the fields of software engineering, artificial intelligence and automotive computing because it integrates web development, intelligent diagnostics, predictive analysis and user-centred interface design into a practical software artefact.DriveSense AI was developed using Python and the Flask framework alongside HTML, CSS and JavaScript for the user interface. The system includes several integrated features including live telemetry monitoring, AI-assisted fault interpretation, predictive maintenance analysis, service record management and PDF diagnostic reporting. The platform also incorporates conversational AI functionality capable of explaining diagnostic faults and maintenance recommendations using natural language. Machine learning-based prediction logic was implemented to estimate vehicle health, repair urgency and safe-to-drive status using diagnostic and telemetry data.

The project aims to improve accessibility and understanding of automotive diagnostics for non-technical users by transforming complex vehicle data into clear and actionable insights. Rather than functioning solely as a fault-code reader, DriveSense AI acts as an intelligent vehicle assistant capable of supporting users with maintenance awareness, diagnostic interpretation and preventative decision-making. The project therefore demonstrates how artificial intelligence can be integrated into automotive software systems to enhance usability, predictive maintenance support and intelligent driver assistance.

Objective	Success Criteria
Develop a functional web-based automotive diagnostic platform	Users can successfully access and navigate all major system features
Implement live diagnostic and telemetry monitoring	Vehicle data is displayed accurately in real time
Integrate AI-assisted fault interpretation	The system generates understandable fault explanations and recommendations
Implement predictive maintenance functionality	Maintenance risk predictions and urgency analysis are generated successfully
Design a responsive and modern user interface	The platform operates effectively across desktop and mobile devices
Evaluate usability and functionality	Testing demonstrates that the system satisfies project aims and operates reliably

3. BACKGROUND AND LITERATURE REVIEW

Modern vehicles have evolved into highly computerised systems containing multiple Electronic Control Units (ECUs), embedded sensors and communication networks that continuously monitor vehicle performance and safety functions. These systems generate large amounts of diagnostic data that can be accessed through On-Board Diagnostics II (OBD-II) technology. OBD-II is a standardised diagnostics protocol designed to monitor engine performance, emissions systems and vehicle faults in real time (SAE International, 2022). As vehicles become increasingly software-driven, there has been growing demand for intelligent diagnostic systems capable of analysing this data more effectively.

Artificial Intelligence (AI), Machine Learning (ML) and predictive maintenance technologies have become important research areas within automotive engineering and computer science. Predictive maintenance refers to the use of intelligent algorithms and data analysis techniques to identify potential failures before breakdowns occur (Zonta et al., 2020). Unlike traditional reactive maintenance approaches, predictive systems aim to improve reliability, reduce repair costs and minimise downtime through continuous monitoring and early fault detection.

Several studies have explored the use of machine learning within predictive maintenance systems. Zonta et al. (2020) explain that machine learning models can analyse large datasets to identify abnormal operating patterns and predict failures. Similarly, Carvalho et al. (2019) identified predictive maintenance as a key Industry 4.0 technology due to its ability to improve operational efficiency and maintenance planning. These studies demonstrate the growing relevance of intelligent monitoring systems across industrial and automotive environments.

The automotive industry has also experienced significant growth in connected vehicle technologies and intelligent diagnostics. Modern vehicles may contain more than 100 ECUs responsible for engine management, braking systems, infotainment and safety controls (Bosch, 2023). This increasing electronic complexity makes vehicle diagnostics more difficult for ordinary users, particularly when diagnostic trouble codes require technical interpretation. Research by Biondi et al. (2019) highlighted the importance of user-centred automotive interfaces capable of simplifying complex diagnostic information and improving accessibility for drivers.

Several commercial diagnostic applications already exist, including Carly, Torque Pro and FIXD. These systems provide features such as fault-code scanning and live telemetry monitoring using Bluetooth OBD-II adapters. However, many existing solutions still rely heavily on technical terminology and provide limited predictive maintenance functionality. In addition, some commercial platforms require subscription payments or proprietary hardware, reducing accessibility for general users.

Research into AI-assisted diagnostics has also increased in recent years. Khan and Salah (2018) discussed the growing integration of AI technologies within Internet of Things (IoT) systems, explaining how connected devices can process large volumes of sensor data in real time. This is particularly relevant to modern vehicles, which operate as IoT environments containing interconnected sensors and control modules. Intelligent analysis of vehicle telemetry data therefore creates opportunities for more advanced diagnostic and maintenance systems.

Conversational AI and natural language processing technologies have further expanded the potential of intelligent automotive systems. AI assistants powered by large language models can explain diagnostic faults, simplify technical terminology and provide maintenance recommendations in natural language. This improves accessibility for non-technical users compared to traditional diagnostic tools that often display raw numerical data and fault codes without explanation.

Despite these developments, several limitations remain within current automotive diagnostic systems. Many applications focus primarily on fault detection rather than predictive analysis, while diagnostic scanning, telemetry monitoring and maintenance management are often separated into different platforms. Existing systems may also provide limited support for responsive design and user-friendly interfaces. Furthermore, fault codes such as “P0301” or “P0171” remain difficult for non-technical users to interpret without specialist knowledge.

The development of DriveSense AI aims to address these limitations by integrating OBD-II diagnostics, predictive maintenance analysis, AI-assisted fault interpretation and responsive interface design into a single web-based platform. The system combines live telemetry monitoring, diagnostic scanning, machine learning prediction models and conversational AI functionality to create a more intelligent and accessible automotive diagnostic solution. Unlike traditional diagnostic tools, DriveSense AI focuses on both technical functionality and user experience, ensuring that diagnostic information remains understandable for ordinary vehicle owners while still providing advanced analytical capabilities.

The literature reviewed within this section demonstrates that AI-driven predictive maintenance systems are becoming increasingly important within the automotive sector. Existing studies support the use of machine learning, IoT integration and intelligent user interfaces for improving maintenance efficiency and fault interpretation. However, current commercial systems still present limitations regarding accessibility, predictive functionality and integrated AI support. Therefore, the development of DriveSense AI represents a relevant contribution within the field of intelligent automotive software systems.

4. METHODOLOGY

This project adopted an iterative and user-focused software development methodology informed by Agile development principles and human-computer interaction approaches (Beck et al., 2001; Sharp, Rogers & Preece, 2019). An iterative methodology was selected because the project combines web development, automotive diagnostics, machine learning concepts and intelligent interface design within a single software platform. Agile principles allowed system features to be developed progressively through continuous implementation, testing and refinement.

The methodology began with background research into OBD-II diagnostics, predictive maintenance systems, conversational AI and existing automotive diagnostic platforms. Commercial applications including Carly, Torque Pro and FIXD were analysed to identify common functionality, interface design approaches and limitations within current automotive diagnostic systems. Academic research relating to predictive maintenance and intelligent vehicle systems was also reviewed to support design and implementation decisions (Zonta et al., 2020).

Following the research stage, functional and non-functional requirements were identified. The primary objectives included live telemetry monitoring, diagnostic fault detection, predictive maintenance analysis, AI-assisted fault interpretation and responsive interface design. These requirements were used to guide implementation and evaluation throughout the project lifecycle.

The design phase focused on system architecture and interface usability. Initial interface layouts and dashboard concepts were designed using Figma before implementation within the Flask application. User-centred design principles were considered to improve accessibility, responsiveness and ease of navigation across desktop and mobile devices (Nielsen, 1994). UML diagrams, activity diagrams and system architecture diagrams were also produced during the design stage to support software planning and system modelling.

The implementation phase involved developing the DriveSense AI platform using Python and the Flask web framework within Visual Studio Code. Flask was selected due to its lightweight architecture and suitability for integrating APIs, machine learning functionality and database operations. HTML, CSS and JavaScript were used to develop the front-end interface, while Python handled server-side logic, telemetry processing and diagnostic analysis.

OBD-II functionality was integrated to retrieve live telemetry data and diagnostic trouble codes from connected vehicles. Diagnostic data was processed using fault interpretation logic to generate understandable fault descriptions, severity levels and maintenance recommendations. Predictive maintenance functionality was implemented using rule-based analysis and machine learning concepts to estimate vehicle health, repair urgency and safe-to-drive status from telemetry and diagnostic data.

Conversational AI functionality was also integrated to provide simplified explanations of vehicle faults and maintenance recommendations through natural language interaction. This improved accessibility by reducing the technical complexity typically associated with automotive diagnostic systems.

The completed system was evaluated through functionality testing, usability-focused testing and interface evaluation methods commonly used within software engineering projects (Sommerville, 2016). Functional testing verified the accuracy of telemetry monitoring, diagnostic scanning and predictive analysis outputs, while usability testing assessed interface responsiveness, navigation and overall user interaction quality across different devices and screen sizes.

Ethical considerations were also considered throughout development. Vehicle data was not stored unnecessarily, and AI-generated recommendations were presented as advisory outputs rather than professional mechanical guarantees. Overall, the methodology provided a structured and practical approach for developing and evaluating the DriveSense AI platform.

5. ANALYSIS/REQUIREMENTS

The analysis and requirements stage focused on identifying the functional, technical and usability requirements necessary for the development of the DriveSense AI platform. This phase established the foundation for the system design and implementation process by analysing existing automotive diagnostic technologies, predictive maintenance research and user interface requirements.The analysis process included reviewing academic research relating to predictive maintenance, artificial intelligence and human-computer interaction. Previous studies demonstrated that predictive maintenance systems can improve maintenance efficiency and reduce failure risks through intelligent monitoring and fault prediction techniques (Zonta et al., 2020). Research into usability engineering also highlighted the importance of accessibility, responsiveness and simplified interfaces within modern software systems (Nielsen, 1994). These findings influenced the development approach of the DriveSense AI platform, particularly regarding AI-assisted diagnostics and user-centred interface design.

Commercial automotive diagnostic applications including Carly, Torque Pro and FIXD were also analysed during the requirements gathering process. These platforms provided examples of vehicle telemetry dashboards, fault-code scanning and diagnostic visualisation features. However, several limitations were identified, including overreliance on technical terminology, limited predictive maintenance functionality and restricted accessibility for non-technical users. Existing systems frequently focused on displaying raw diagnostic information rather than providing intelligent interpretation or maintenance guidance. Practical experimentation using OBD-II diagnostic tools and telemetry simulation was also conducted to evaluate how vehicle data could be retrieved, processed and displayed within a web-based environment. Telemetry values such as RPM, coolant temperature, fuel level and engine load were examined to determine how predictive maintenance analysis could be implemented effectively. Flask development and AI integration experiments were additionally performed to evaluate the suitability of Python-based technologies for the project. The information collected during analysis was converted into functional and non-functional requirements. Functional requirements defined the core system features including diagnostic scanning, telemetry monitoring, predictive maintenance analysis, AI-assisted fault interpretation and report generation. Non-functional requirements focused on system usability, responsiveness, reliability and cross-device compatibility (Sommerville, 2016).

User interface analysis also formed an important part of the requirements process. Human-computer interaction principles discussed by Sharp, Rogers and Preece (2019) were considered throughout development to ensure the platform remained understandable and accessible for users with limited automotive knowledge. As a result, the final interface incorporated responsive layouts, simplified navigation and colour-coded diagnostic indicators to improve readability and usability.

Overall, the analysis stage demonstrated the need for an automotive diagnostic platform that combines technical functionality with intelligent interpretation and accessibility. These findings directly influenced the development of DriveSense AI as an integrated web-based system capable of supporting diagnostics, predictive maintenance and AI-assisted vehicle analysis.

5.1 Functional Requirements

Requirement ID	Requirement Description	Priority
FR1	The system shall allow users to register and securely log into the application	High
FR2	The system shall connect to an OBD-II diagnostic device	High
FR3	The system shall retrieve and display live telemetry data including RPM, coolant temperature and engine load	High
FR4	The system shall scan and display diagnostic trouble codes (DTCs)	High

FR5	The system shall provide understandable explanations for fault codes	High
FR6	The system shall generate AI-assisted diagnostic recommendations	High
FR7	The system shall calculate predictive maintenance risks using telemetry and fault data	High
FR8	The system shall display vehicle health scores and severity indicators	Medium
FR9	The system shall allow users to manage service records and maintenance history	Medium
FR10	The system shall generate downloadable diagnostic reports in PDF format	Medium
FR11	The system shall support AI-based vehicle image analysis	Medium
FR12	The system shall support responsive layouts across desktop and mobile devices	High

5.2 Non-Functional Requirements

Requirement ID	Requirement Description	Priority
NFR1	The system shall provide a responsive and user-friendly interface	High
NFR2	The platform shall display telemetry information with minimal delay	High
NFR3	The system shall maintain stable performance during diagnostic scanning	High

NFR4	The application shall support secure user authentication	High
NFR5	The platform shall support modern web browsers and mobile devices	Medium
NFR6	The system shall maintain readable and accessible interface layouts	Medium
NFR7	The software architecture shall support modular expansion and future feature development	Medium
NFR8	AI-generated recommendations shall be presented clearly and understandably	Medium

6. ARTEFACT DESIGN AND IMPLEMENTATION

6.1 Artefact Overview

The artefact developed for this project is DriveSense AI, a web-based intelligent automotive diagnostic and predictive maintenance platform designed to assist vehicle owners in monitoring vehicle health, diagnosing faults and understanding maintenance risks through artificial intelligence and OBD-II telemetry data. The system combines real-time vehicle diagnostics, predictive maintenance analysis, AI-assisted fault interpretation and responsive user interface design within a single integrated application.

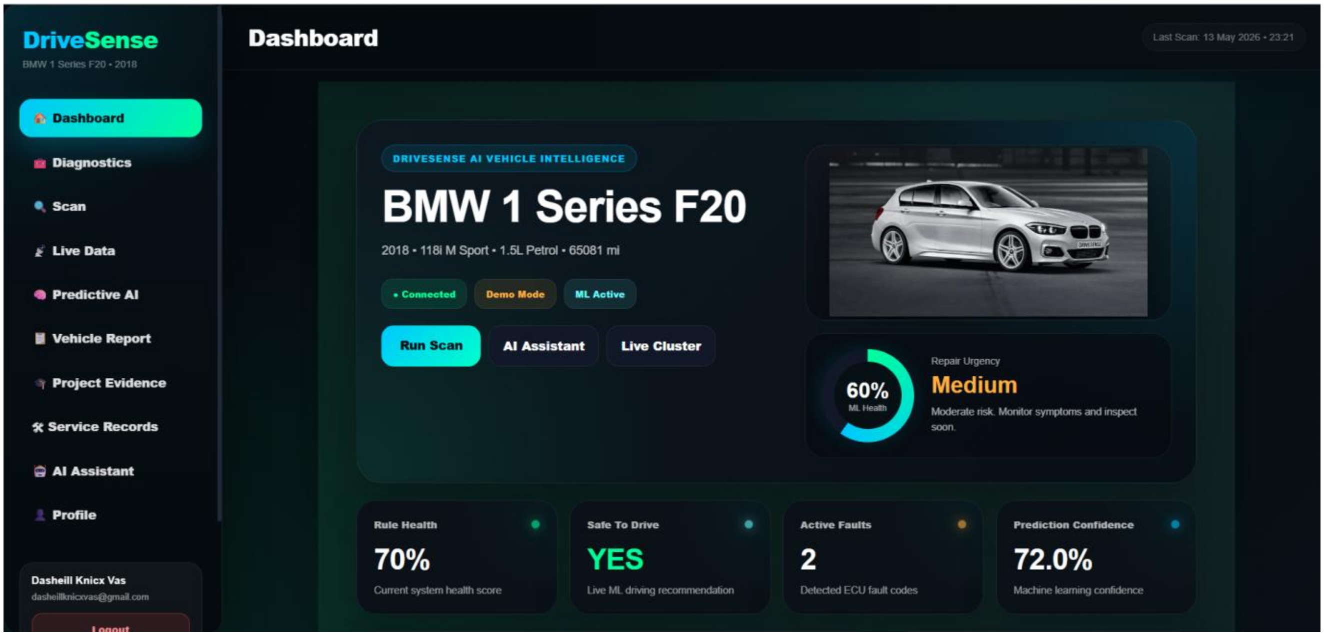
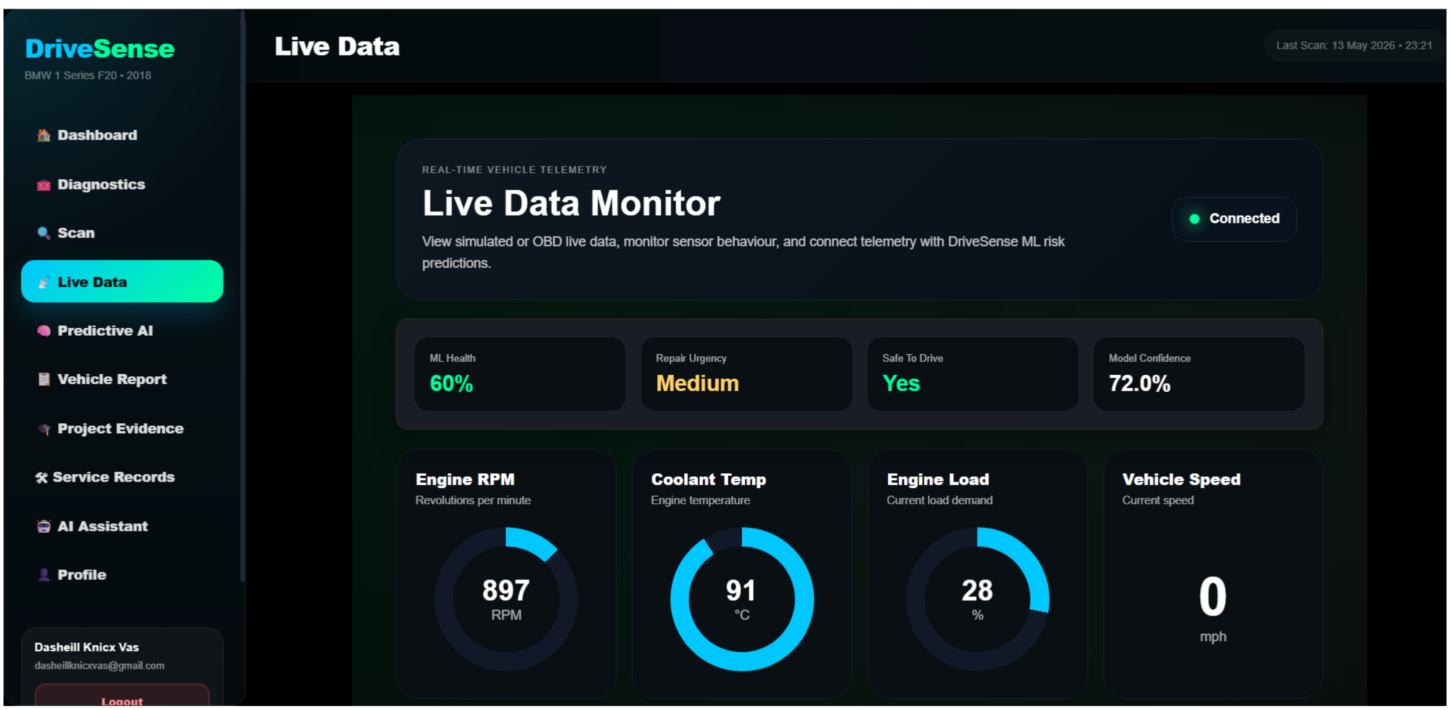


Figure 1 DriveSense AI main dashboard interface providing access to diagnostics, predictive maintenance analysis, live telemetry monitoring and AI-assisted vehicle support features.

Extended explanation provided in Appendix I.



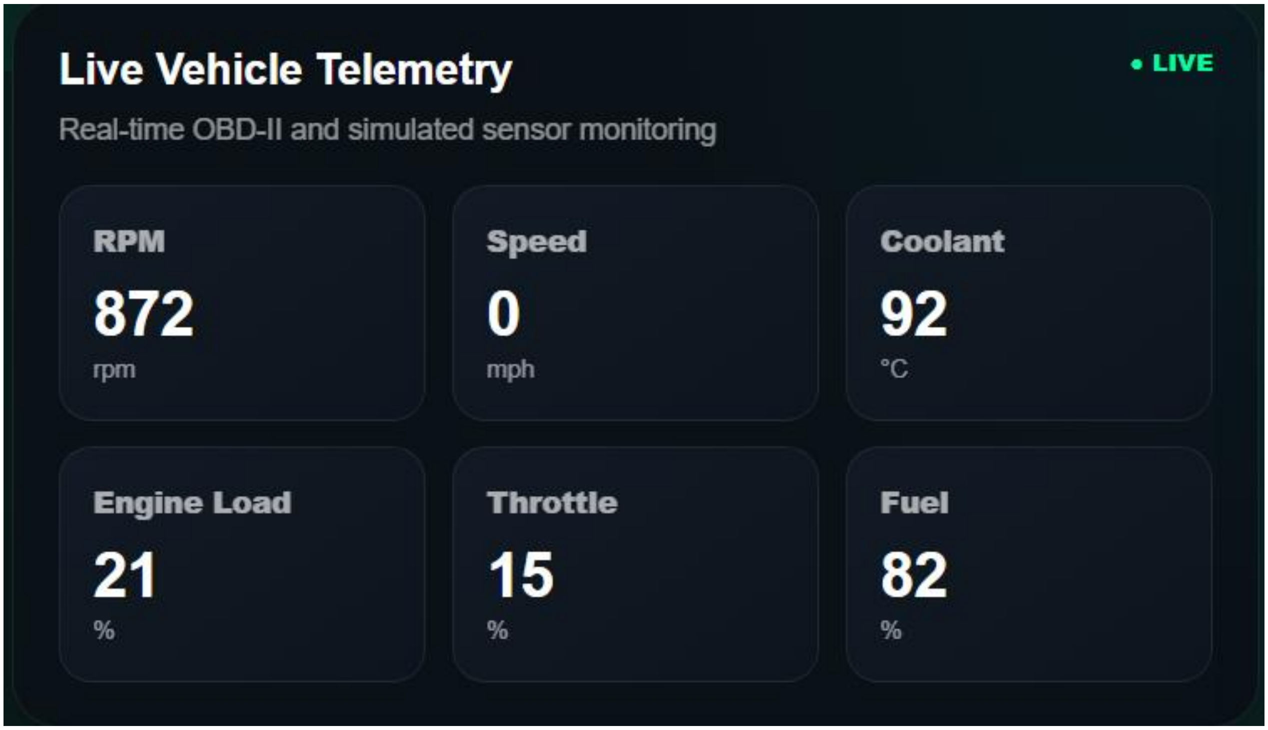


Figure 2 DriveSense AI live vehicle telemetry interface displaying real-time OBD-II sensor data including RPM, speed, coolant temperature, throttle position and fuel level during diagnostic monitoring.

Extended explanation provided in Appendix I.

The overall architecture of the system follows a modular software design approach. The application consists of separate components responsible for authentication, diagnostics, telemetry monitoring, predictive maintenance, AI analysis, report generation and service record management. This modular structure improved maintainability, scalability and debugging efficiency during development (Sommerville, 2016).

The outputs gathered during the analysis and requirements stage directly influenced the design and implementation process. The identified requirement for improved accessibility and ease of use led to the development of a dashboard-oriented user interface with simplified navigation and visual diagnostic indicators. Existing commercial systems were analysed, and several usability limitations were identified, particularly regarding technical terminology and poor user guidance. As a result, DriveSense AI was designed to prioritise understandable AI explanations and simplified vehicle health summaries.

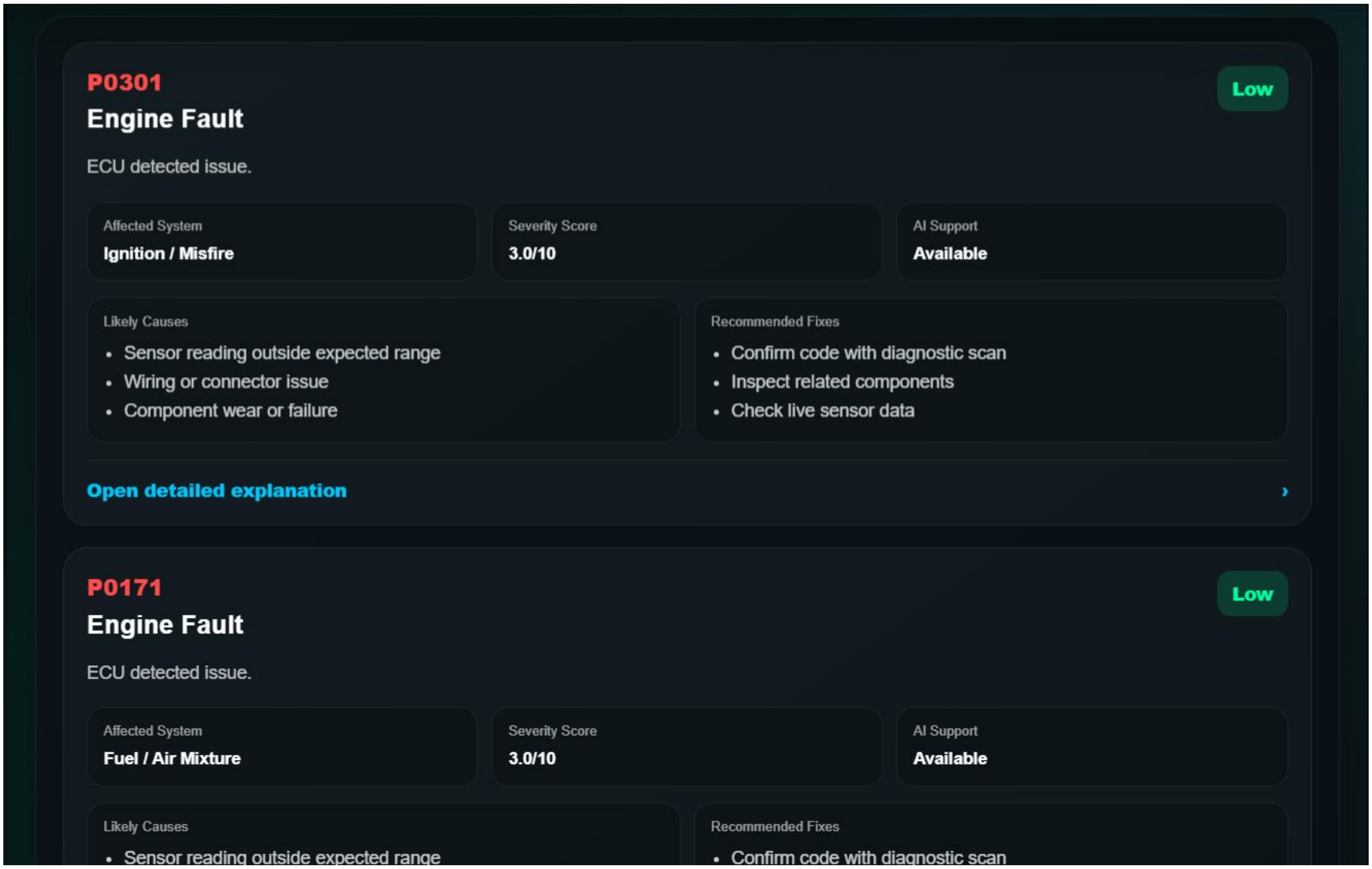


Figure 3 DriveSense AI fault analysis interface displaying diagnostic trouble codes, severity scoring and AI-assisted repair recommendations during vehicle diagnostics.

Extended explanation provided in Appendix I.

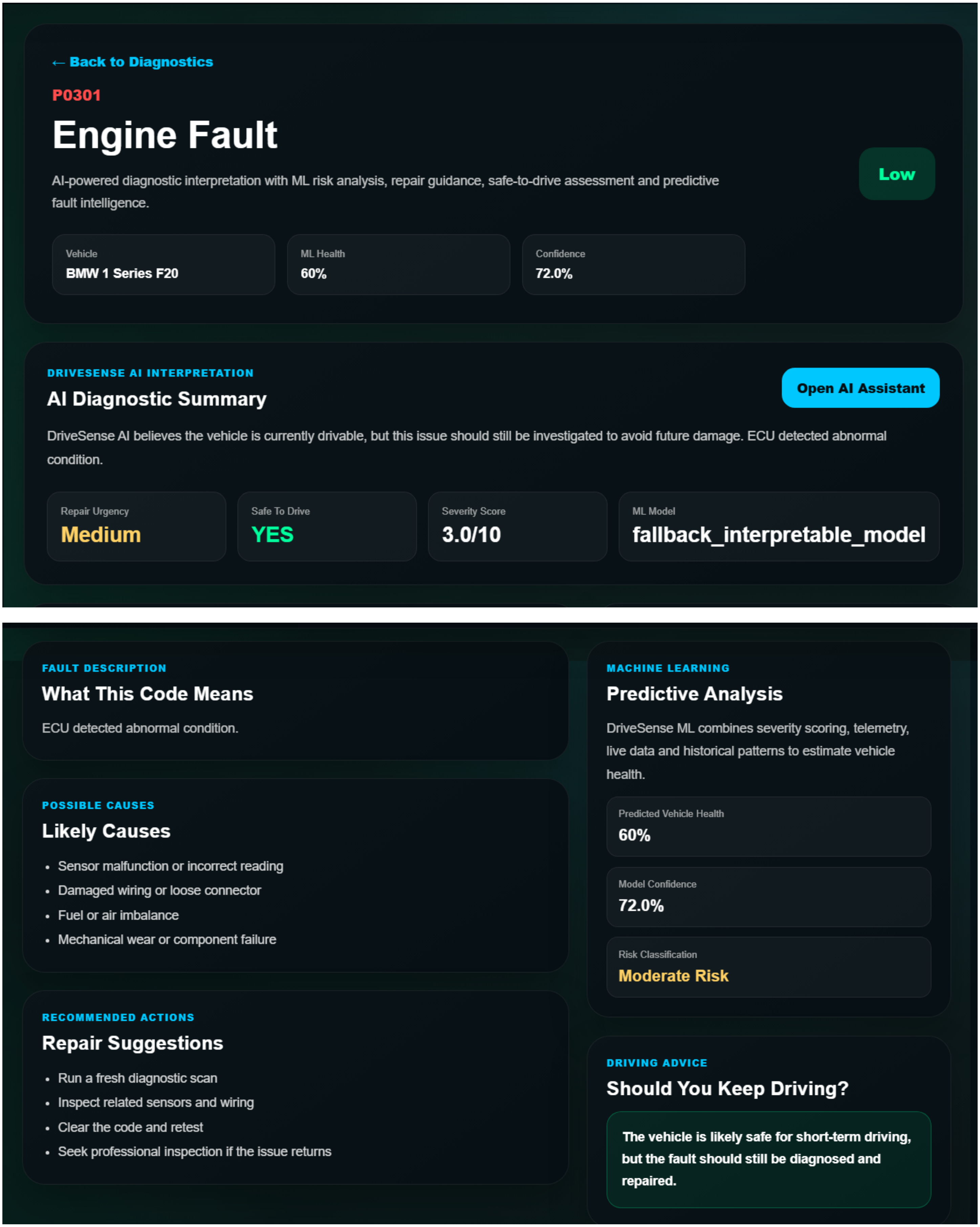


Figure 4 DriveSense AI fault analysis and AI recommendation interface presenting predictive vehicle health assessment, repair suggestions and safe-to-drive guidance during diagnostic analysis.

Extended explanation provided in Appendix I.

The implementation process was completed incrementally using an iterative development approach. The first stage involved creating the Flask application structure, folder organisation and routing system. Blueprint routing was implemented to separate application functionality into manageable modules. Authentication functionality was then implemented using secure login and registration systems supported by session management. This allowed user-specific vehicle data and scan history to be securely stored throughout the application lifecycle.

The next stage focused on user interface development. A responsive layout was designed using HTML and CSS with dark-mode inspired dashboard styling. The interface included a sidebar navigation system, responsive telemetry cards, vehicle status indicators and mobile-friendly layouts. Human-computer interaction principles discussed by Sharp, Rogers and Preece (2019) influenced several interface decisions including consistent navigation, reduced interface clutter and clear visual feedback mechanisms.

After the interface foundation was completed, vehicle diagnostic functionality was implemented. OBD-II integration services were developed to retrieve telemetry data such as RPM, coolant temperature, throttle position, fuel level and engine load. Diagnostic trouble code scanning functionality was then integrated using Python-based automotive diagnostic services. Retrieved DTC codes were linked to a local diagnostic database which allowed the system to provide understandable fault descriptions, severity analysis and repair recommendations.

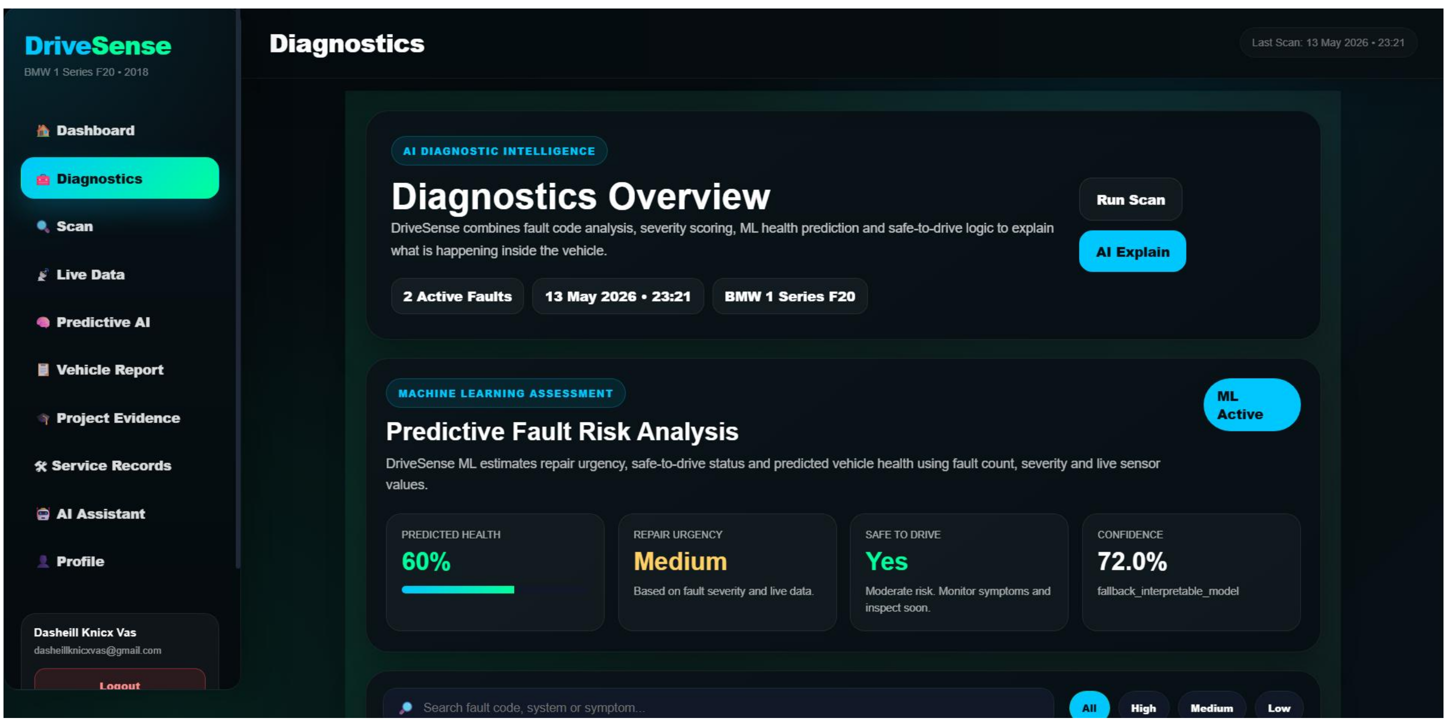


Figure 5 DriveSense AI Diagnostics Interface

Extended explanation provided in Appendix I.

Predictive maintenance functionality represented one of the most innovative aspects of the project. Machine learning-inspired logic was implemented to calculate vehicle health scores, maintenance urgency and predictive risk levels based on telemetry values and detected faults. The predictive system analysed live vehicle data and combined it with severity scoring algorithms to estimate maintenance urgency and safe-to-drive conditions. The implementation was influenced by predictive maintenance models discussed in Industry 4.0 research literature (Zonta et al., 2020).

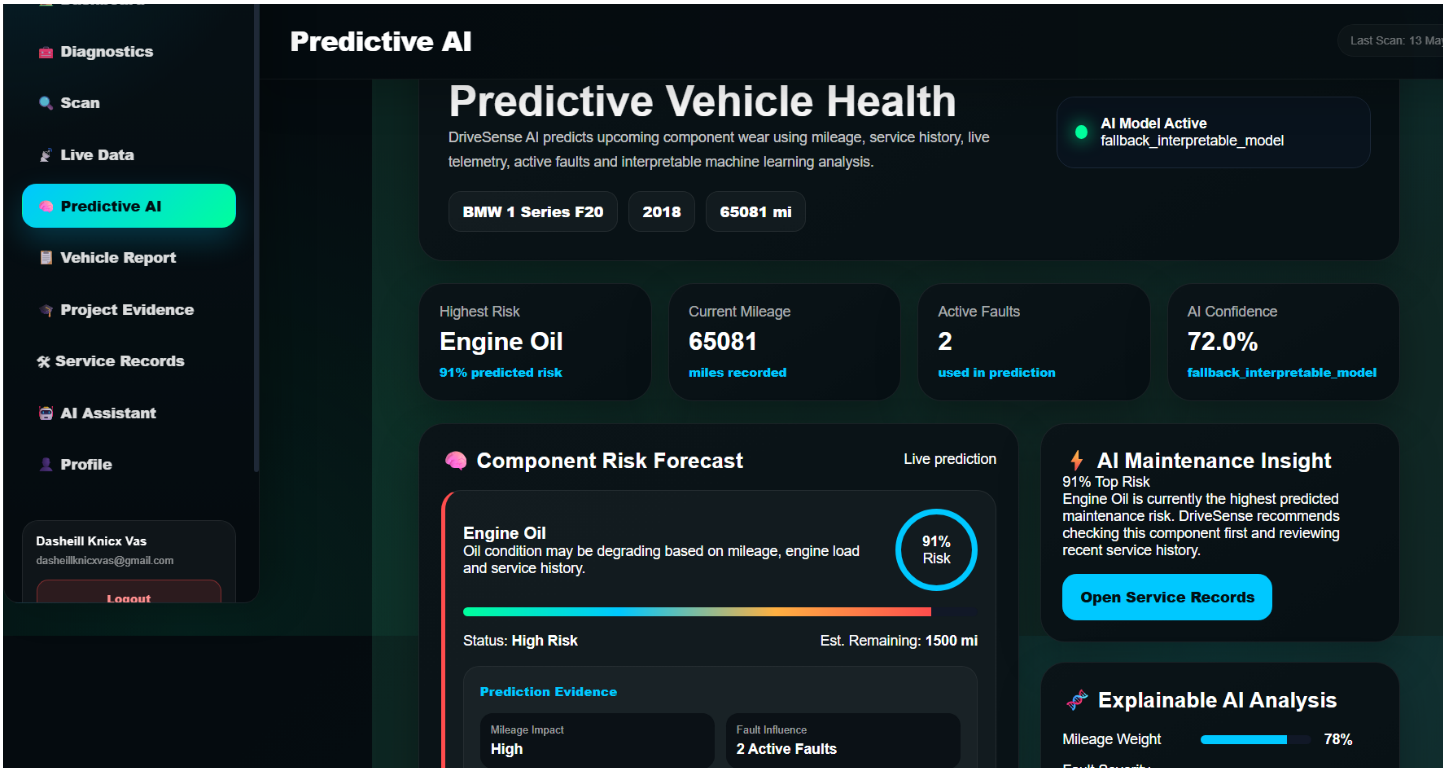


Figure 6 DriveSense AI Predictive Maintenance and AI Analytics Interface

Extended explanation provided in Appendix I.

AI-assisted diagnostic interpretation was also implemented as a key feature of the artefact. Conversational AI integration allowed users to ask questions regarding detected faults and receive understandable explanations and recommendations. The AI system used contextual prompts that included live telemetry data, fault information and predictive maintenance outputs to generate personalised responses. This functionality improved accessibility for users with limited automotive knowledge. An additional innovative feature included image-based vehicle analysis using AI image recognition models. Users were able to upload images of vehicle components or dashboard warning lights for automated analysis. The AI system examined uploaded images and generated explanations regarding visible mechanical issues or dashboard warnings. This functionality extended the diagnostic capability of the platform beyond traditional telemetry-based systems.

Several technical challenges were encountered during implementation. One significant challenge involved maintaining stable communication between the Flask application and locally hosted AI models. Timeout handling, API request management and error recovery systems were implemented to improve reliability. Another challenge involved ensuring that the user interface remained responsive across desktop and mobile devices. Extensive CSS adjustments and responsive layout testing were required to maintain consistent visual behaviour across different screen sizes. The implementation process also involved the development of PDF report generation functionality. The system was capable of generating downloadable diagnostic reports containing vehicle details, telemetry information, predictive maintenance summaries, fault analysis and service history data. Report Lab was integrated within the Flask application to dynamically generate structured PDF reports.

Overall, the final artefact successfully combined automotive diagnostics, predictive maintenance, artificial intelligence and responsive web technologies within a unified platform. The implementation demonstrated how AI and telemetry systems can be integrated into accessible automotive software capable of assisting users in understanding vehicle health and maintenance requirements more effectively than traditional diagnostic tools.

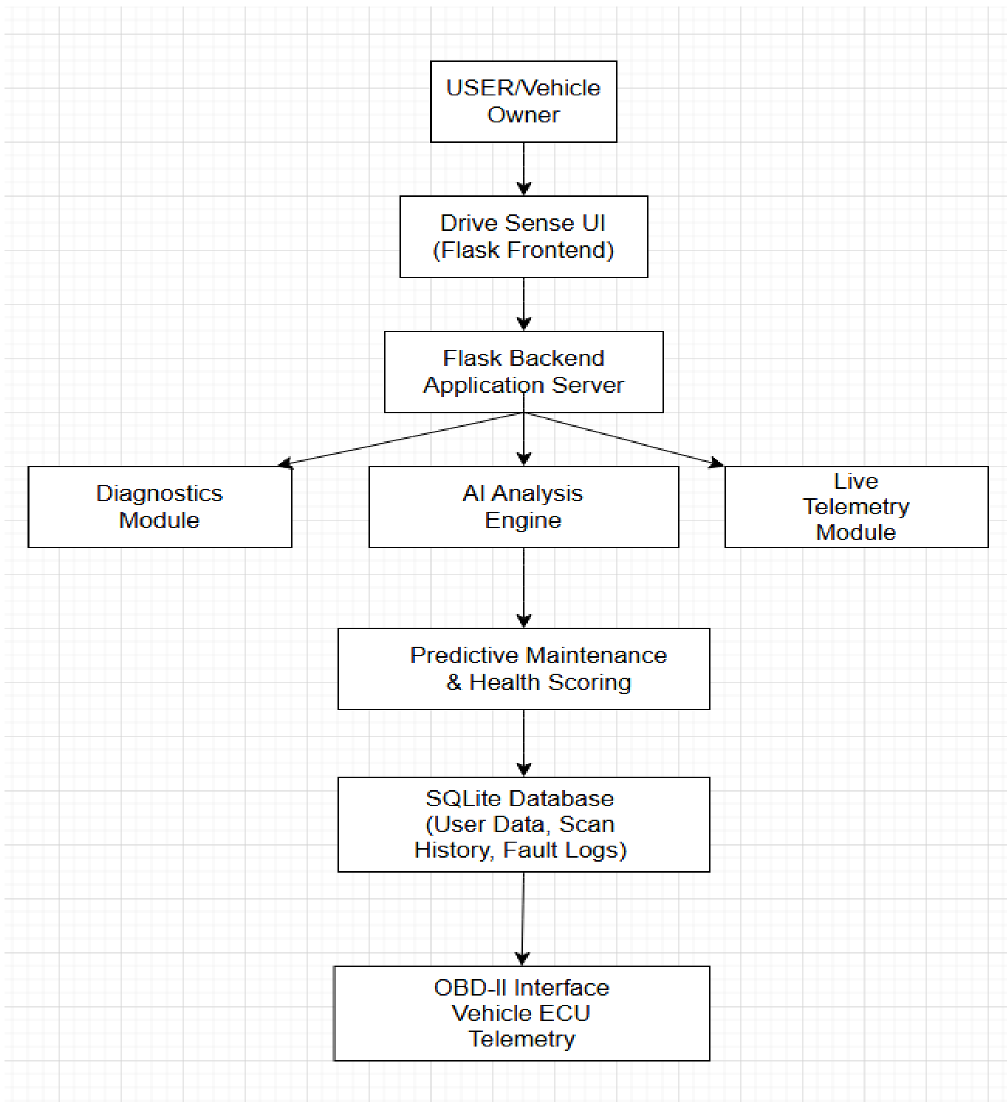


Figure 7 DriveSense AI System Architecture Diagram

Figure 7 DriveSense AI system architecture diagram illustrating how the user interface communicates with the Flask backend, AI analysis engine, telemetry modules and SQLite database to process vehicle diagnostics, predictive maintenance analysis and real-time OBD-II vehicle data.

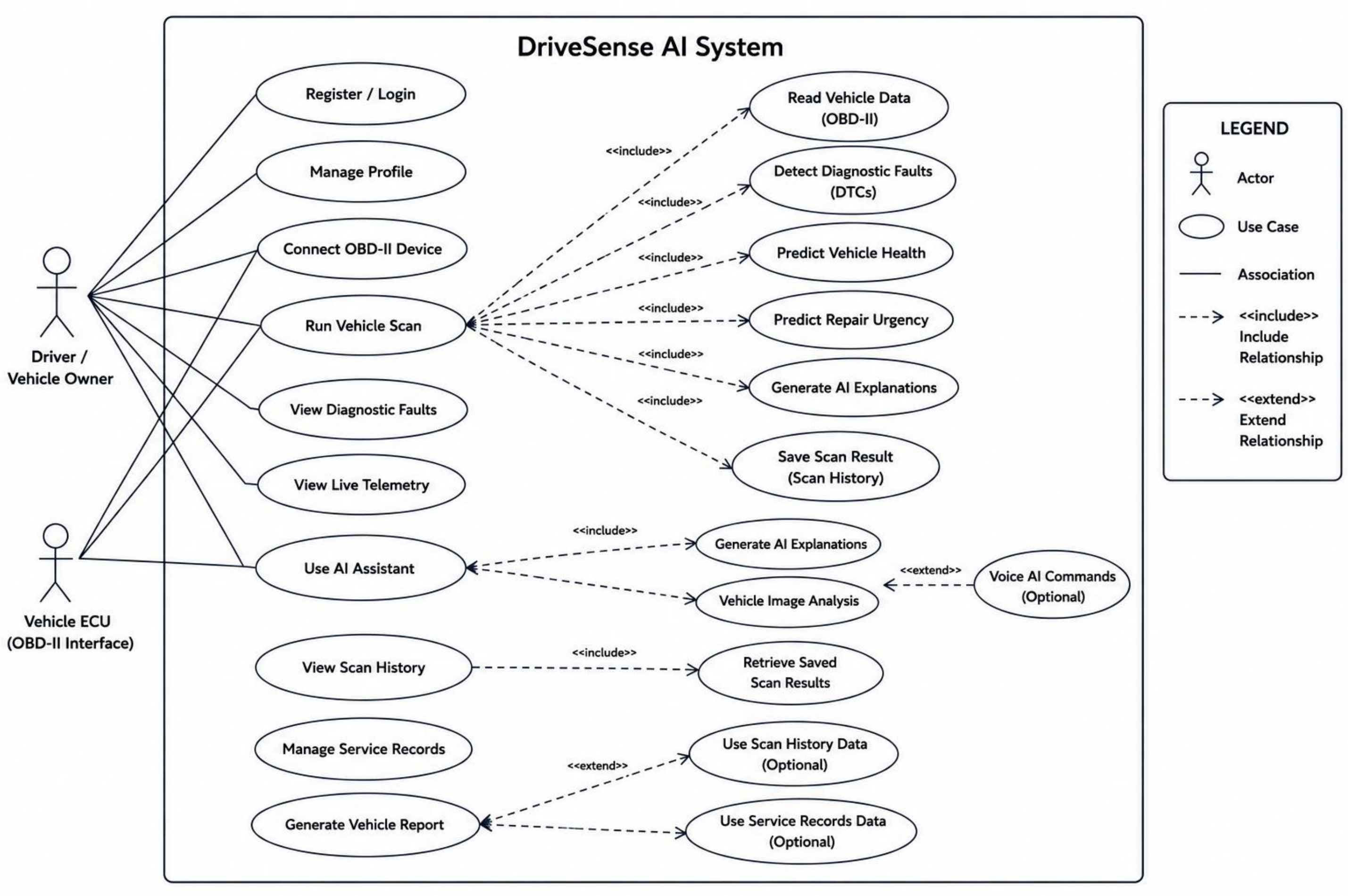


Figure 8. UML Use Case Diagram of the DriveSense AI system showing user interactions, vehicle communication processes and AI-assisted diagnostic functionalities.

6.2 Use Case Specification – Run Vehicle Scan

Field	**Description**
Use Case Name	Run Vehicle Scan
Primary Actor	Driver / Vehicle Owner
Supporting Actor	Vehicle ECU / OBD-II Interface
Goal	To scan the vehicle, retrieve diagnostic information, analyse vehicle condition and display results to the user.
Preconditions	The user is logged into DriveSense AI. The vehicle diagnostic interface is connected or demo mode is available.
Trigger	The user selects the “Start Scan” or “Run Scan” option within the DriveSense AI application.
Main Success Scenario	1. The user opens the scan page.2. The user starts the diagnostic scan.3. The system checks the vehicle connection.4. The system retrieves live telemetry data.5. The system detects diagnostic trouble codes.6. The system calculates vehicle health and repair urgency.7. The system generates AI-assisted diagnostic recommendations.8. The system saves the scan result.9. The scan summary is displayed to the user.
Alternative Flow	If the OBD-II connection is unavailable, the system uses demo/simulated diagnostic data to allow the scan workflow to continue for testing and demonstration purposes.
Exception Flow	If scan processing fails, the system displays an error message and allows the user to restart the scan.

Postconditions	The user can view scan results, detected faults, health score, repair urgency and saved scan history.
Related Requirements	FR2, FR3, FR4, FR5, FR6, FR7, FR8
Priority	High

This use case was selected because it represents the core workflow of the DriveSense AI platform, combining vehicle communication, telemetry processing, diagnostic scanning, AI-assisted analysis and scan history storage.

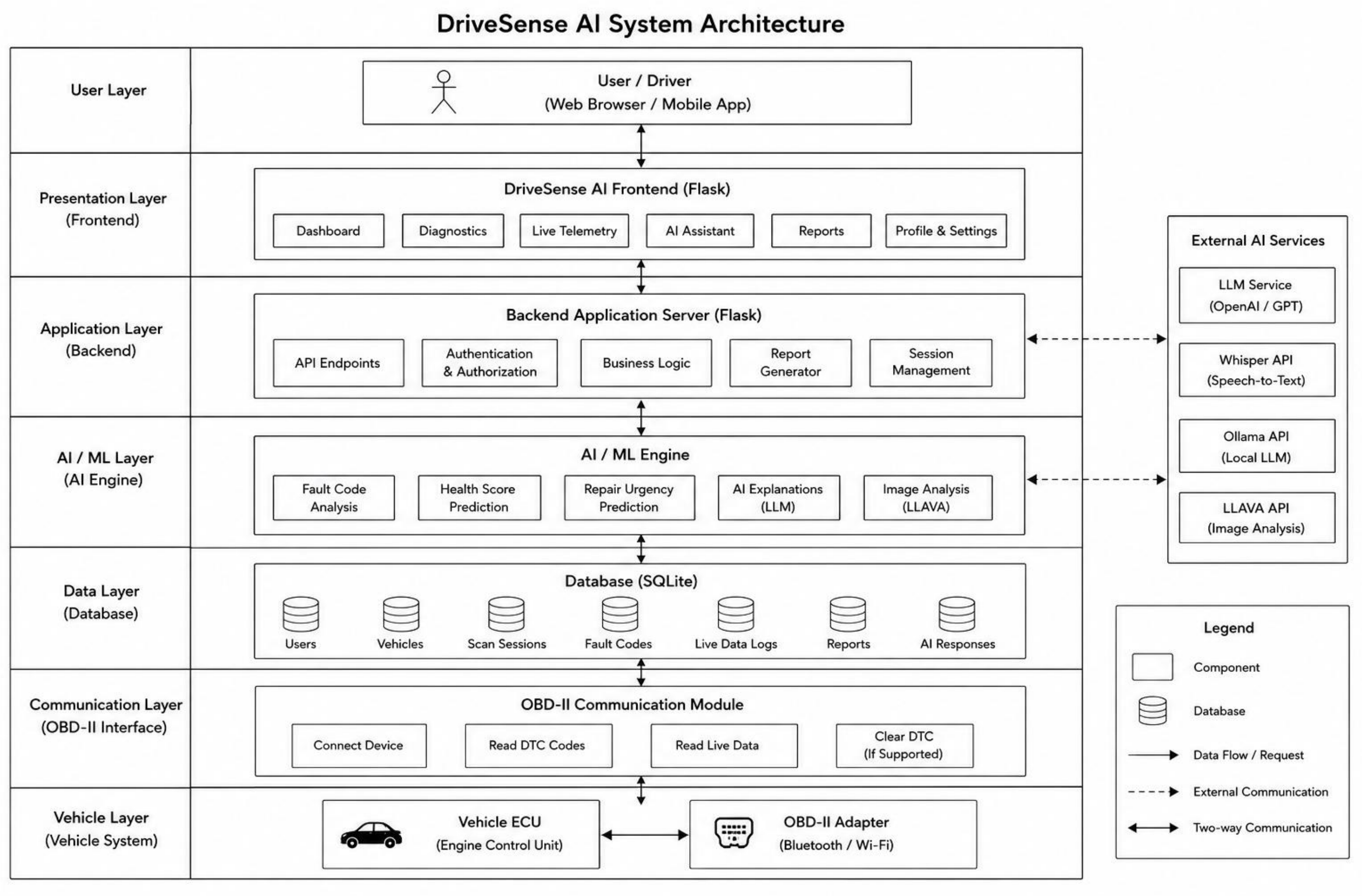


Figure 9 DriveSense AI System Diagram

This system architecture diagram illustrates the overall structure and data flow of the DriveSense AI platform. The architecture is divided into multiple layers to separate user interaction, application processing, artificial intelligence functionality, database management and vehicle communication.The user interacts with the system through the frontend interface, which provides access to features such as diagnostics, live telemetry, AI assistance and report generation. Requests are processed by the Flask backend application server, which manages authentication, business logic and communication between system components.The AI and machine learning layer performs intelligent analysis including fault-code interpretation, vehicle health prediction, repair urgency estimation and AI-generated explanations. Diagnostic and telemetry data are stored within the SQLite database layer for scan history, reporting and analysis purposes.The communication layer handles OBD-II interaction by retrieving live vehicle telemetry and diagnostic trouble codes from the vehicle ECU through an OBD-II adapter. External AI services such as OpenAI, Ollama, Whisper and LLAVA are integrated to provide conversational AI, speech processing and image analysis functionality.Overall, the architecture demonstrates how DriveSense AI combines automotive diagnostics, artificial intelligence and web technologies into a unified intelligent vehicle maintenance platform.

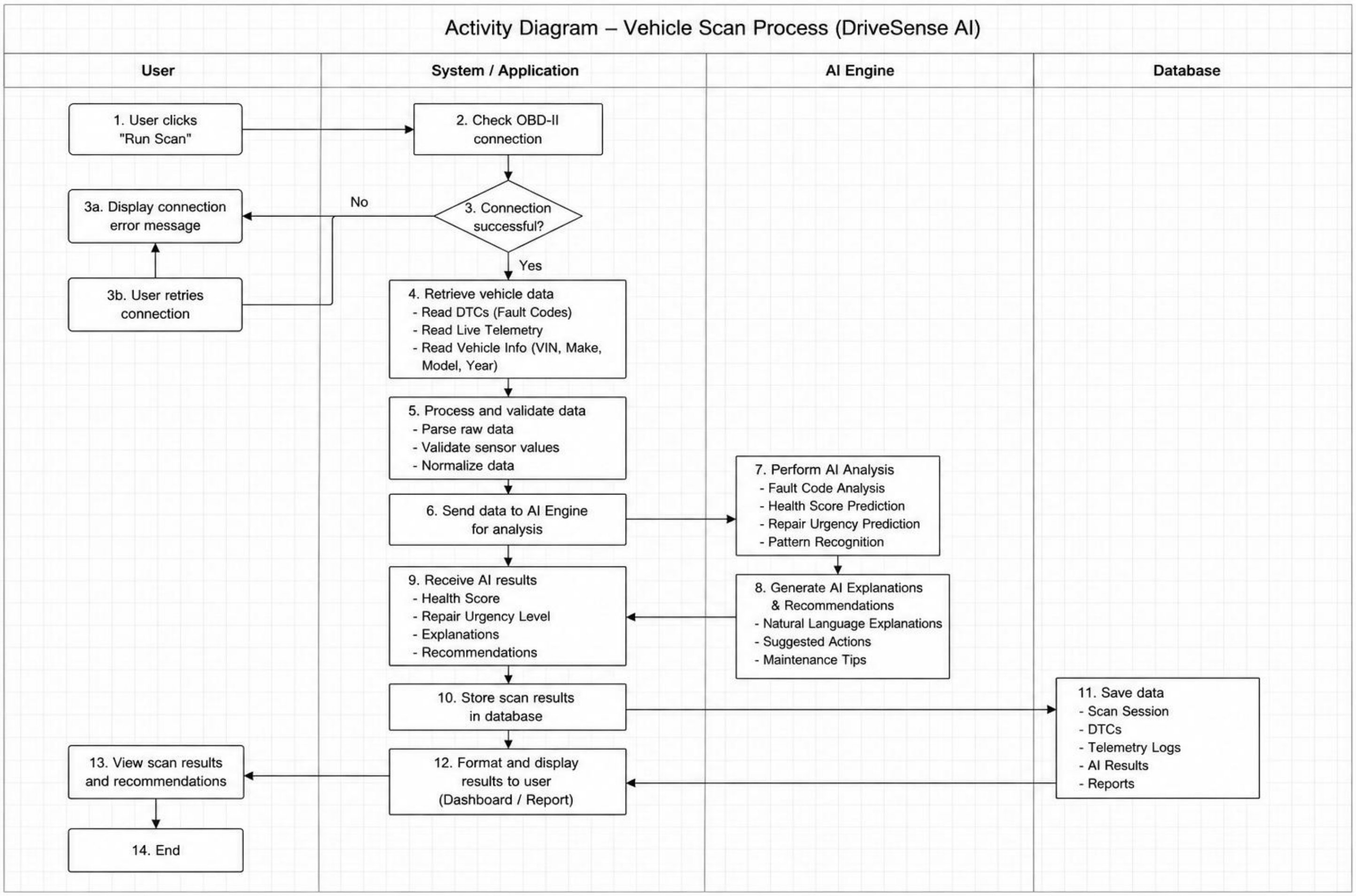


Figure 10 Activity Diagram of the DriveSense AI Vehicle Scan and AI Analysis Process

Figure 10 illustrates the overall vehicle scan and AI analysis workflow within the DriveSense AI platform. The process begins when the user initiates a vehicle scan through the application interface. The system first validates the OBD-II connection before retrieving diagnostic trouble codes (DTCs), live telemetry data and vehicle information. The collected data is then processed and sent to the AI engine for predictive maintenance analysis, fault interpretation and recommendation generation. Finally, the analysed results are stored in the database and displayed to the user through the dashboard and diagnostic report interface.

7. Evaluation

The evaluation stage was conducted to assess the functionality, usability and overall effectiveness of the DriveSense AI platform. The purpose of the evaluation was to determine whether the developed artefact successfully achieved the objectives identified during the analysis and requirements stage. Particular focus was placed on diagnostic functionality, predictive maintenance analysis, interface usability and system responsiveness. Evaluation is an important component of software engineering because it allows systems to be assessed against both technical and user-centred requirements (Sommerville, 2016).

The evaluation process included functional testing, usability-focused testing and comparative analysis. Functional testing was performed throughout development to verify that major system components operated correctly. Features including live telemetry monitoring, diagnostic fault scanning, AI-assisted fault interpretation, predictive maintenance analysis and PDF report generation were tested individually before integration into the final platform. Black-box testing techniques were used to compare system outputs against expected behaviour. Diagnostic trouble codes were tested to ensure that appropriate fault descriptions, severity ratings and maintenance recommendations were generated correctly.

Usability evaluation focused on determining whether users could navigate the platform effectively and understand the information presented within the interface. Human-computer interaction principles including consistency, visibility and user feedback were considered during testing (Nielsen, 1994). Particular attention was given to whether non-technical users could interpret AI-generated explanations and predictive maintenance outputs without specialist automotive knowledge.

The evaluation involved six participants with varying levels of automotive and technical experience. Participants were asked to complete a series of tasks including navigating the dashboard, viewing telemetry data, performing diagnostic scans and accessing predictive maintenance information. Feedback was then collected regarding usability, design clarity and overall system interaction.

The evaluation findings indicated that the platform successfully achieved the majority of its objectives. Participants reported that the interface was modern, visually clear and easy to navigate across desktop and mobile devices. Users also responded positively to the AI-assisted fault explanations, stating that the system simplified complex automotive terminology into more understandable recommendations. These findings support research suggesting that user-centred interfaces improve accessibility and usability within technical systems (Sharp, Rogers & Preece, 2019).

Functional testing demonstrated that the core system components operated reliably. Live telemetry values including RPM, coolant temperature and engine load updated successfully within the dashboard interface. Diagnostic fault scanning correctly returned fault information and associated recommendations, while predictive maintenance analysis generated repair urgency levels and vehicle health scores using telemetry and diagnostic data. PDF report generation functionality also operated successfully during testing.

Despite the positive outcomes, several limitations were identified. AI-generated analysis occasionally experienced minor processing delays due to the computational requirements of locally hosted AI models. In addition, OBD-II communication reliability depended on hardware compatibility and stable vehicle connections during testing. These limitations highlighted the importance of future optimisation and broader real-world evaluation.

Several opportunities for future improvement were also identified during evaluation. Suggested enhancements included mobile application deployment, cloud-based data storage, expanded predictive maintenance models and integration with manufacturer-specific diagnostic databases. Participants also suggested additional voice interaction functionality to improve accessibility. Research indicates that predictive maintenance systems become increasingly effective when trained using larger telemetry datasets (Zonta et al., 2020).

Overall, the evaluation demonstrated that DriveSense AI successfully integrated automotive diagnostics, predictive maintenance and AI-assisted analysis within a responsive web-based platform. The system provided understandable diagnostic information, real-time telemetry monitoring and intelligent maintenance recommendations that improved accessibility for non-technical users. The findings therefore indicate that the project represents a successful implementation of intelligent automotive diagnostic technologies within a software engineering context.

7.1 Evaluation Findings table

Evaluation Area	Findings
User Interface	Users reported that the interface was modern, responsive and easy to navigate
Diagnostic Scanning	Fault code retrieval and explanations operated correctly
AI Assistant	Participants found AI explanations understandable and useful
Predictive Maintenance	Maintenance urgency predictions operated successfully
Mobile Responsiveness	Application functioned correctly across multiple screen sizes
Report Generation	PDF reports generated successfully with accurate information
Limitations	Minor AI response delays and dependency on OBD hardware compatibility

7.2 System Performance Evaluation

The overall performance of the DriveSense AI platform was evaluated based on responsiveness, functionality, usability and integration of AI-assisted automotive diagnostic features. The system was successfully implemented as a Flask-based web application capable of displaying vehicle telemetry data, performing diagnostic scans and generating AI-assisted recommendations through an integrated interface.

The dashboard interface demonstrated stable performance during testing and was capable of updating live telemetry values such as RPM, speed, coolant temperature and engine load dynamically without requiring manual page refreshes. Real-time updates improved user interaction and provided a more realistic automotive monitoring experience.

The predictive maintenance and health scoring components successfully generated repair urgency predictions and vehicle health estimations using rule-based analysis combined with machine learning concepts. The AI assistant functionality was also capable of generating simplified explanations of fault codes and maintenance recommendations to improve accessibility for non-technical users.

The user interface was designed with a responsive layout to support both desktop and smaller screen devices. Testing confirmed that the application remained visually consistent across multiple display sizes while maintaining functionality and usability.

Overall, the system achieved the primary objectives defined during the requirements and analysis stages by integrating diagnostics, telemetry monitoring, predictive analysis and AI-assisted fault interpretation into a single intelligent automotive platform.

7.3 Functional Testing

Functional testing was conducted throughout the development process to ensure that the core components of the DriveSense AI platform operated correctly and satisfied the defined system requirements. Testing focused on validating the functionality of diagnostic scanning, AI-assisted analysis, telemetry monitoring, report generation and user interface interactions.

The diagnostic scanning functionality was tested to verify that the system could successfully retrieve and display vehicle fault codes through the OBD-II interface. The application correctly identified active diagnostic trouble codes and displayed severity levels alongside AI-generated explanations and repair recommendations.

Live telemetry functionality was also evaluated to ensure that vehicle data such as RPM, vehicle speed, coolant temperature, throttle position and fuel level updated dynamically within the dashboard interface. Repeated testing confirmed stable refresh behaviour and accurate data visualisation during simulated and live sessions.

The AI assistant module was tested using multiple diagnostic scenarios and user queries. The system successfully generated contextual responses, maintenance advice and simplified explanations designed to improve accessibility for non-technical users. Additional testing verified that predictive maintenance calculations and vehicle health scoring components generated appropriate outputs based on detected vehicle conditions.

User interface testing was performed across different screen resolutions to evaluate responsiveness and layout consistency. The system maintained stable navigation behaviour and adaptive layouts across desktop and smaller display environments.

7.4 Functional Testing Table

Feature Tested	Expected Result	Actual Result	Status
OBD-II Vehicle Scan	Retrieve diagnostic fault codes	Fault codes displayed successfully	Pass
Live Telemetry	Real-time vehicle data updates	Telemetry updated dynamically	Pass

AI Assistant	Generate diagnostic explanations	AI responses generated correctly	Pass
Maintenance	Calculate health predictions	Predictions displayed correctly	Pass
Vehicle Report Generation	Generate scan reports	Reports created successfully	Pass
Responsive Interface	Adapt to multiple screen sizes	Layout adjusted correctly	Pass

Results for the DriveSense AI Platform. It summarises the primary functional testing outcomes achieved during evaluation.

7.5 Practical Vehicle Testing and Hardware Validation

In addition to simulated diagnostic testing, practical real-world validation was conducted using a BMW 1 Series F20 vehicle connected through an ENET Ethernet-to-OBD diagnostic interface. The testing process evaluated hardware communication, vehicle telemetry access, diagnostic interaction and practical deployment of the DriveSense AI platform within a real automotive environment. The practical testing stage demonstrated successful physical integration between the vehicle, OBD-II communication hardware and the diagnostic software environment. Real vehicle service information, telemetry interaction and hardware communication workflows were successfully validated during testing.

Test Area	Validation Outcome	Status
ENET OBD-II Connection	Vehicle communication established successfully	PASS
Ethernet Network Detection	Windows detected active ENET adapter connection	PASS
Diagnostic Communication	Diagnostic software successfully communicated with vehicle ECU systems	PASS
Vehicle Service History Access	BMW service history records displayed correctly	PASS
Real Vehicle Deployment	DriveSense AI testing environment operated successfully inside vehicle	PASS
Hardware Stability	Stable communication maintained during testing session	PASS

Figure 11 Practical deployment of the DriveSense AI testing environment on the BMW F20 vehicle.

Extended explanation provided in Appendix I.

Extended explanation provided in Appendix I.

Figure 12 ENET Ethernet-to-OBD diagnostic interface connected to the BMW OBD-II port.

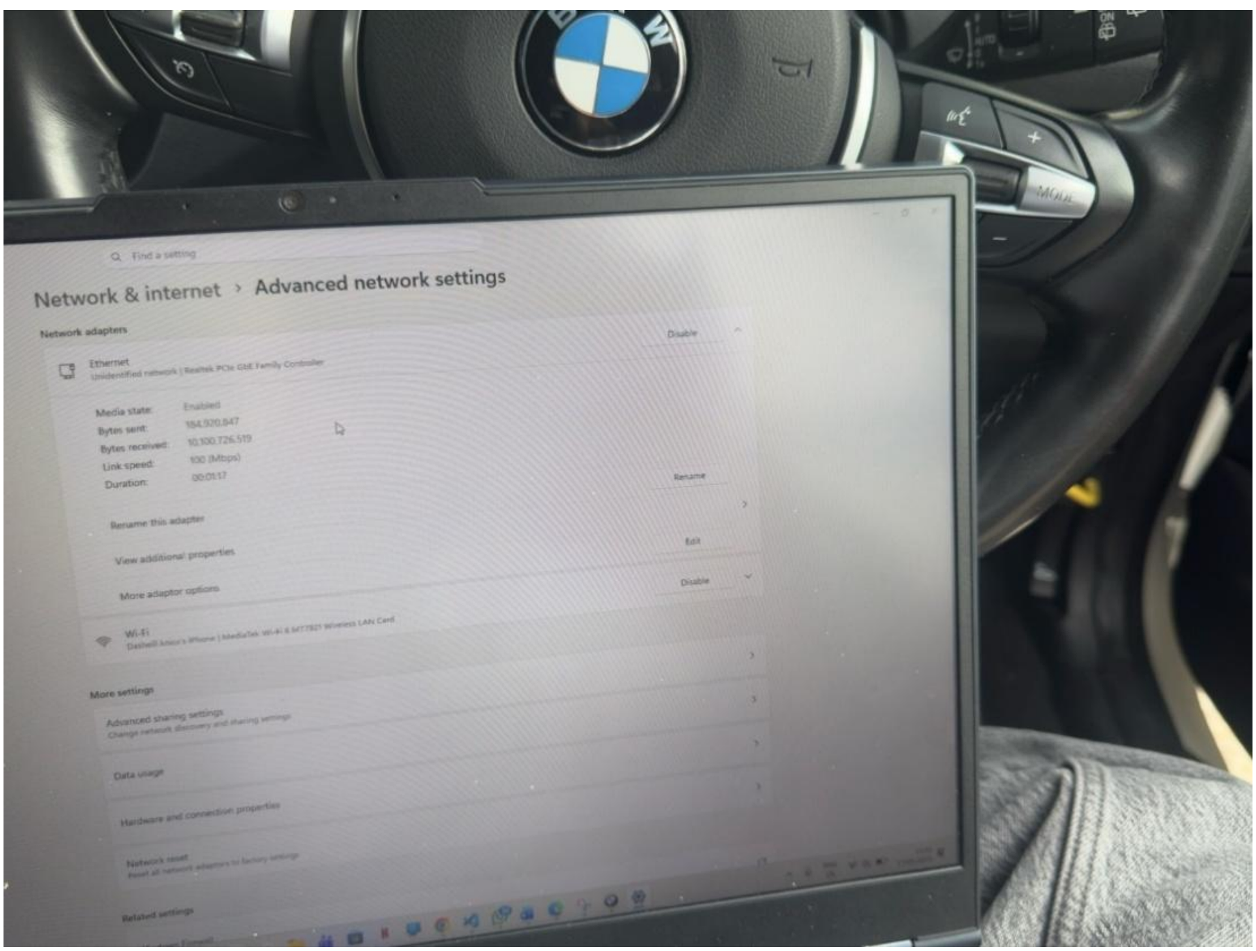
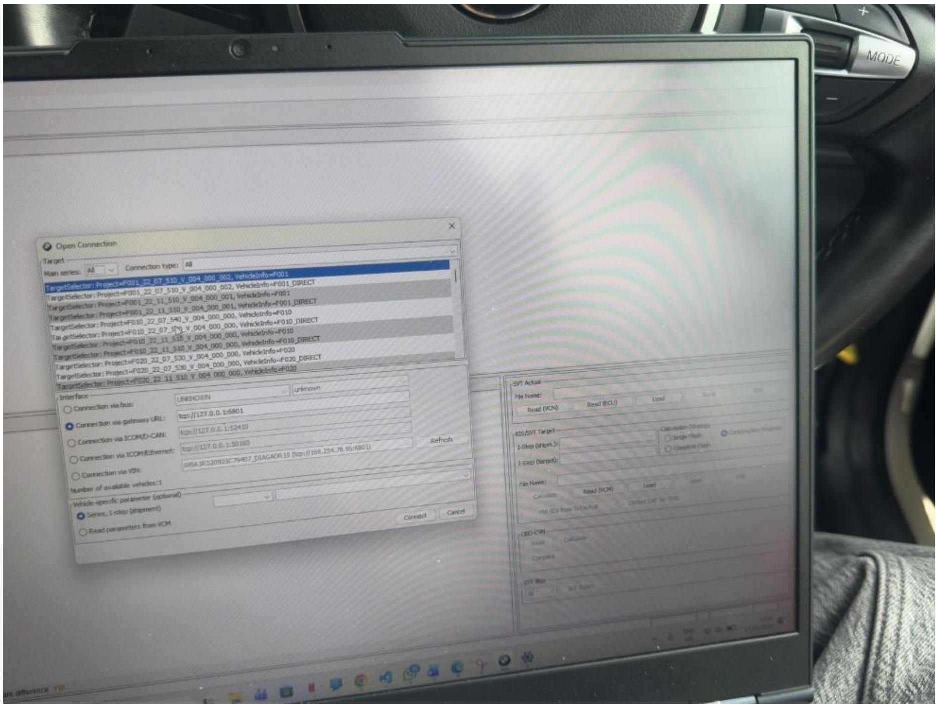


Figure 13 Active Ethernet network communication established during vehicle diagnostic testing.

Extended explanation provided in Appendix I.



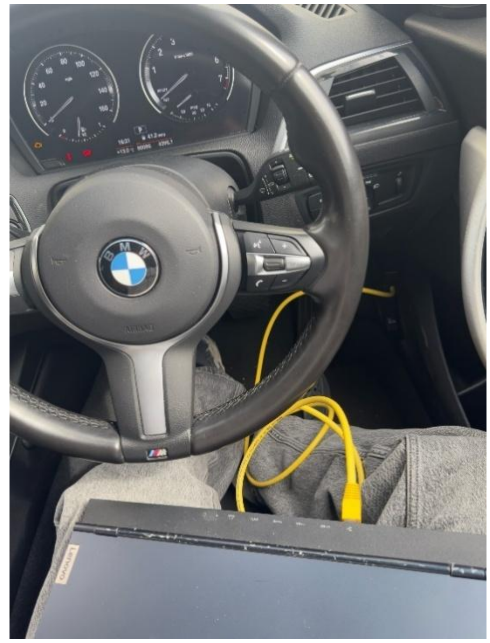


Figure 14 E-Sys ECU communication interface used during BMW diagnostic testing.

Extended explanation provided in Appendix I.

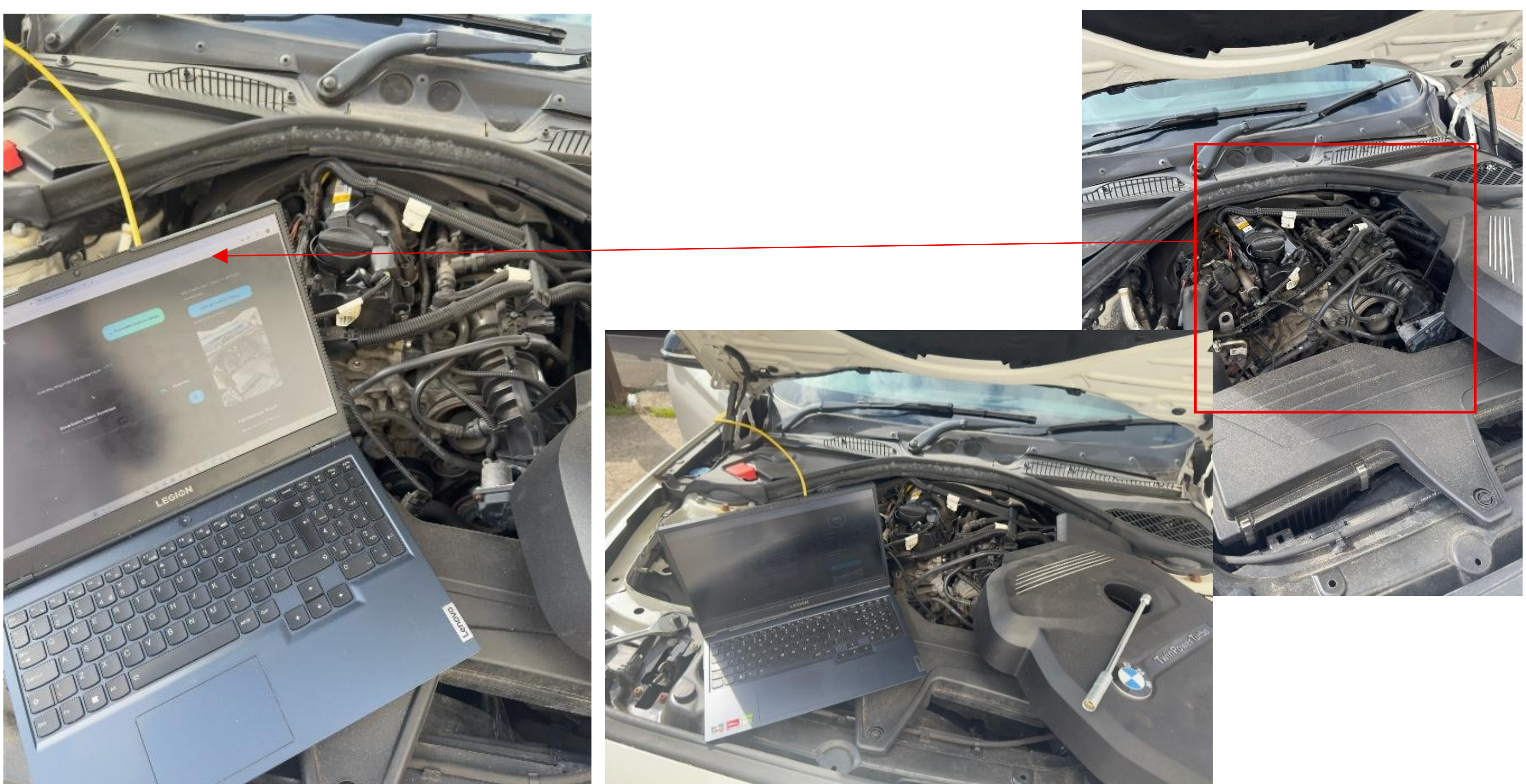


Figure 15 Real-world DriveSense AI validation and live in-vehicle diagnostic testing environment.

Extended explanation provided in Appendix I.

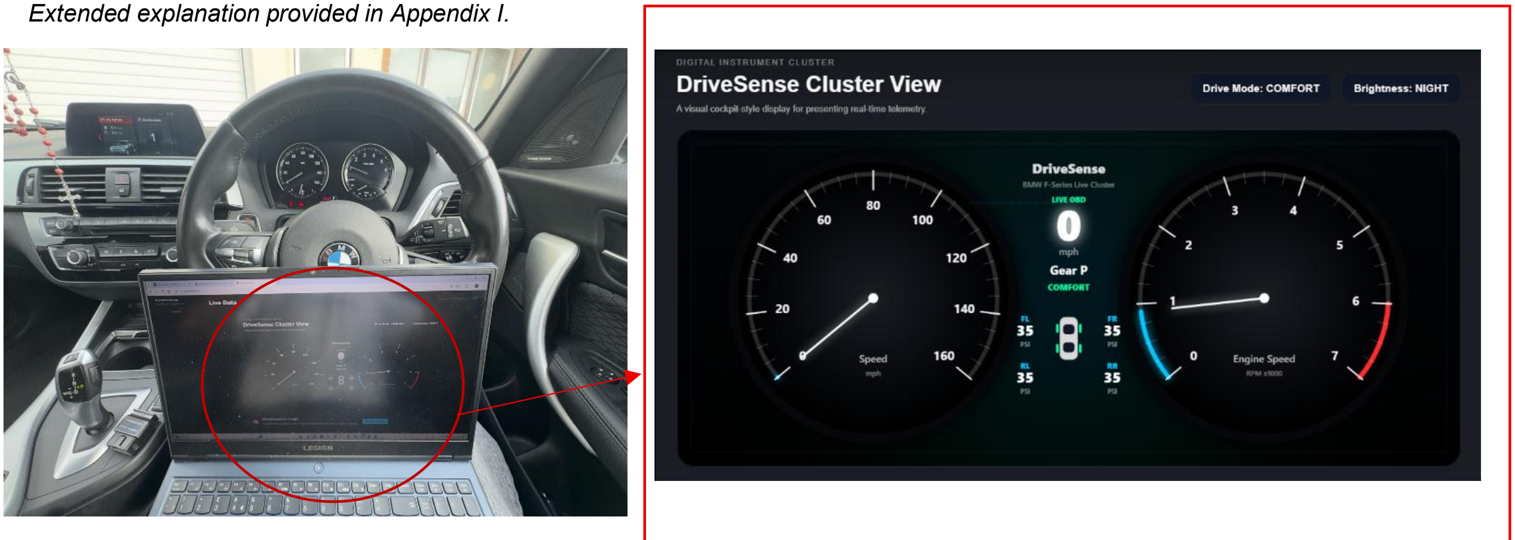


Figure 16 Live DriveSense AI dashboard and real-time vehicle telemetry during practical testing

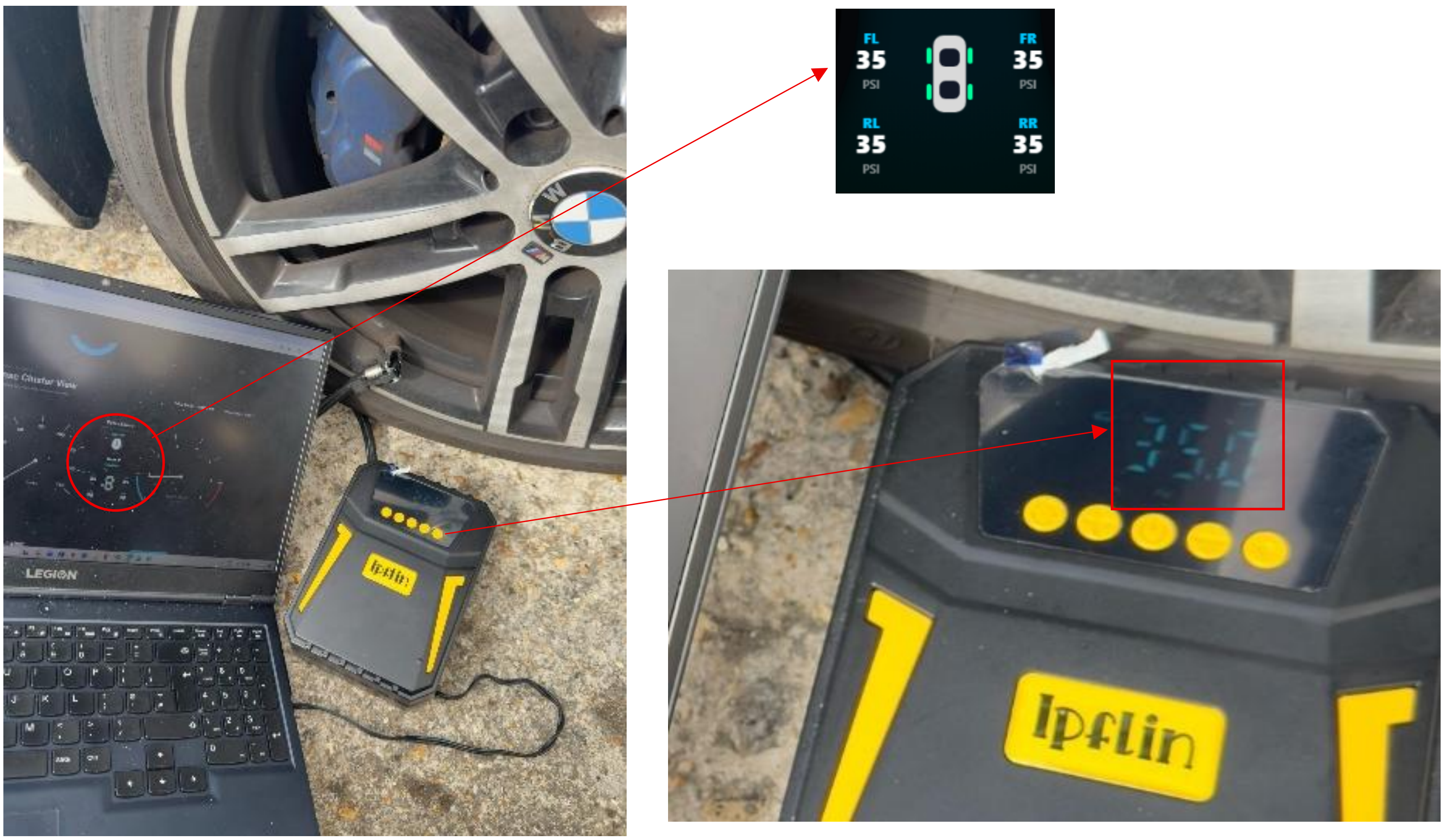


Figure 17 Real-time tyre pressure telemetry validation between the BMW vehicle and DriveSense AI platform.

Extended explanation provided in Appendix I.

Figure 18 AI-assisted repair guidance validated using real workshop tooling during practical testing.

Extended explanation provided in Appendix I.

Figure 19 Vehicle service record management interface within the DriveSense AI platform.

Extended explanation provided in Appendix I.

7.6 Limitations of the System

Although the DriveSense AI platform successfully achieved the primary objectives of the project, several limitations were identified during development and evaluation. These limitations mainly relate to hardware availability, real-world data collection constraints and the complexity of integrating artificial intelligence within automotive diagnostic environments. One limitation of the system was the restricted availability of real vehicle testing environments during development. While the application could communicate with OBD-II systems and processing telemetry data, some testing scenarios relied partially on simulated or demonstration data to validate interface behaviour and predictive analysis functionality.

The predictive maintenance and AI analysis components were implemented using lightweight machine learning and rule-based approaches due to limited access to large-scale automotive datasets. As a result, prediction accuracy may vary depending on vehicle condition, driving behaviour and the availability of historical diagnostic information. More advanced predictive models would require significantly larger training datasets and extended real-world testing. Another limitation relates to compatibility across different vehicle manufacturers and ECU implementations. Although OBD-II provides standardised communication protocols, some vehicle-specific diagnostic information may not be fully accessible through generic interfaces. This may affect the availability of advanced telemetry values or manufacturer-specific fault codes.

The AI assistant component also depends on the quality of the underlying language model and diagnostic interpretation logic. While the system successfully generated simplified explanations and recommendations, AI-generated responses may occasionally require further verification by experienced automotive technicians before real-world repair decisions are made. Finally, the current implementation primarily focuses on desktop web deployment. Although the interface was designed responsively, the system has not yet been fully optimised for dedicated mobile application deployment or cloud-based scalability.

8. Conclusion

This project successfully designed and developed DriveSense AI, an intelligent automotive diagnostic and predictive maintenance platform that combines OBD-II telemetry, machine learning concepts and conversational artificial intelligence within a responsive web-based system. The project addressed the growing demand for accessible and understandable automotive diagnostic technologies by developing a platform capable of monitoring vehicle health, interpreting diagnostic faults and supporting predictive maintenance analysis for users with varying levels of automotive knowledge.

The completed artefact demonstrated how modern software engineering approaches can be effectively applied within automotive diagnostic environments to improve both functionality and accessibility. Using Flask, Python and responsive web technologies, the platform successfully integrated live telemetry monitoring, diagnostic fault scanning, AI-assisted fault interpretation and predictive maintenance analysis within a single unified application. The project also demonstrated the practical implementation of intelligent automotive software systems, supporting existing research highlighting the increasing importance of AI-driven predictive maintenance technologies within modern transportation systems (Zonta et al., 2020; Lee et al., 2014).

Several strengths were identified throughout the development and evaluation process. One significant strength involved the successful integration of multiple technologies including telemetry monitoring, predictive maintenance functionality and conversational AI support within a cohesive software platform. The responsive dashboard interface and simplified fault explanations also improved usability for non-technical users, demonstrating the importance of user-centred design principles within modern technical systems (Sharp, Rogers and Preece, 2019). In addition, the modular application architecture improved maintainability, scalability and future extensibility throughout development.

Despite the successful implementation, several limitations were identified during testing and evaluation. System performance and telemetry reliability depended heavily on OBD-II hardware compatibility and stable vehicle communication. AI-assisted analysis occasionally introduced processing delays due to the computational requirements associated with locally hosted AI models. Furthermore, the predictive maintenance functionality relied primarily on lightweight rule-based and simplified machine learning logic rather than large-scale industrial automotive datasets, limiting prediction accuracy within broader real-world deployment scenarios. These limitations reflect challenges commonly identified within intelligent predictive maintenance research and automotive AI integration studies (Carvalho et al., 2019).

The project also highlighted multiple opportunities for future development and continued research. Future implementations could include cloud-based infrastructure, larger machine learning training datasets, manufacturer-specific ECU integration and dedicated mobile application deployment. Additional enhancements such as voice-controlled interaction, advanced telemetry analytics and real-time cloud synchronisation could further improve usability and intelligent vehicle support capabilities. Future predictive maintenance systems may also benefit from deeper neural network integration and large-scale automotive telemetry modelling approaches identified within recent intelligent maintenance research (Zonta et al., 2020).

Overall, the project successfully achieved its primary aims and objectives by developing an intelligent automotive diagnostic platform that combines vehicle telemetry, predictive maintenance functionality and AI-assisted analysis within a modern software engineering solution. The final artefact provides a strong foundation for future research and development within intelligent automotive software systems while demonstrating the growing role of artificial intelligence within modern vehicle diagnostics and predictive maintenance technologies.

8.1 Future Work

Although the DriveSense AI platform successfully achieved the primary project objectives, several opportunities remain for future development and expansion. Future implementations could include full integration with physical OBD-II hardware devices and manufacturer-specific ECU communication systems to improve real-world diagnostic compatibility and telemetry accuracy.

Further development could also involve cloud-based infrastructure and remote vehicle telemetry storage to support scalable multi-vehicle monitoring environments. Additional machine learning training datasets and more advanced predictive analytics models could improve prediction reliability and maintenance forecasting accuracy, particularly within large-scale automotive diagnostic scenarios.

Dedicated mobile application deployment could further improve accessibility and usability across different platforms. Additional features such as voice-controlled AI interaction, enhanced cybersecurity protection and advanced real-time ECU analytics may also improve intelligent diagnostic capabilities and overall user experience.

Future intelligent automotive systems may increasingly rely on large-scale AI-driven maintenance analytics and connected telemetry ecosystems, supporting previous research regarding the growing role of predictive maintenance technologies within modern transportation environments (Zonta et al., 2020; Lee et al., 2014).

8.2 Achievement and Evaluation of Project Objectives

Original Objective	Outcome Achieved	Evidence
Develop a functional automotive diagnostics platform	Successfully achieved	Functional Flask-based platform demonstrated throughout Chapters 5–7
Integrate OBD-II vehicle communication	Successfully achieved	Practical ENET Ethernet-to-OBD testing demonstrated in Figures 11–14
Implement live vehicle telemetry monitoring	Successfully achieved	Real-time telemetry dashboard demonstrated in Figures 16–17
Develop AI-assisted fault code explanations	Successfully achieved	AI diagnostic explanation functionality demonstrated in Figures 3, 4 and 18
Integrate predictive maintenance functionality	Partially achieved	Predictive maintenance analytics implemented using rule-based and AI-assisted analysis
Design a responsive and modern user interface	Successfully achieved	Responsive desktop and mobile interfaces demonstrated throughout implementation chapter
Evaluate system usability and functionality through testing	Successfully achieved	Practical testing and evaluation discussed in Chapter 7

8.3 Limitations and Future Development Opportunities

Current Limitation	Impact on System	Potential Future Improvement
Predictive maintenance relied on lightweight machine learning and rule-based logic	Reduced prediction accuracy in complex real-world scenarios	Integrate larger automotive datasets and advanced deep learning models
Partial reliance on simulated telemetry data during testing	Limited validation under large-scale real-world driving conditions	Full integration with live OBD-II vehicle telemetry hardware
Local AI model processing introduced response delays	Reduced real-time responsiveness during AI-assisted analysis	Implement cloud-based AI processing and optimisation
Current platform is prototype-scale only	Limited scalability for enterprise or fleet deployment	Develop scalable cloud infrastructure and distribute telemetry systems
Limited manufacturer-specific ECU compatibility	Reduced support across different vehicle brands	Expand ECU protocol integration and manufacturer-specific diagnostics
Web-based deployment only	Reduced portability for mobile users	Develop dedicated Android and iOS mobile applications
Limited cybersecurity implementation	Potential risks within connected automotive environments	Integrate enhanced encryption and automotive cybersecurity frameworks
Limited historical training data	Reduced predictive maintenance reliability	Implement continuous telemetry learning and long-term data collection

The identified limitations provide opportunities for future research and continued development within intelligent automotive diagnostic systems. Addressing these areas could significantly improve predictive maintenance accuracy, system scalability, AI responsiveness and real-world automotive integration capabilities.

9 Acknowledgments

I would like to express my sincere gratitude to my dissertation supervisor, **Dr Emili Balaguer-Ballester** for their continuous guidance, valuable feedback and support throughout the development of this project. Their academic advice, encouragement and expertise were extremely important during both the research and implementation stages of this dissertation.

I would also like to thank the lecturers and staff within the School of Computing & Engineering at Bournemouth University for providing the knowledge and learning opportunities that supported this project.

Special thanks are extended to my family and friends for their patience, encouragement, and ongoing support throughout my studies. Their motivation helped me remain focused and committed during the completion of this dissertation.

I would also like to thank the individuals who participated in the testing and evaluation of the DriveSense AI platform. Their feedback contributed significantly towards improving the usability, functionality, and overall quality of the system.

Finally, I would like to acknowledge the open-source software communities, academic researchers, and developers whose tools, frameworks and published studies supported the successful development of this project.

10 AI Tools and Generative AI Usage

DriveSense AI was developed with limited assistance from Generative AI tools including ChatGPT during selected stages of the project lifecycle. AI-assisted tools were primarily used to support brainstorming, interface planning, documentation refinement, debugging support, architectural planning and improvement of written clarity throughout the dissertation process. ChatGPT was also used to assist in generating early design ideas for user interface layouts, UML diagram planning and organizational structure within the report. Suggestions generated through AI tools were critically reviewed, modified and adapted before inclusion within the final system implementation and dissertation. All software implementation, testing, debugging, evaluation, system integration and final technical decisions were completed independently by the author. AI-generated suggestions were never accepted without manual verification, testing and refinement to ensure technical accuracy and project relevance. Additional tools such as Grammarly and Microsoft Word Editor were used for minor spelling, grammar and readability improvements throughout the report preparation process.

11. References

Biondi, F., Alvarez, I., & Jeong, K. A. (2019). Human-vehicle interaction and automobile interfaces: A review. International Journal of Human-Computer Interaction.

Bosch. (2023). Automotive electronics and ECU systems overview. Bosch Automotive Handbook.

Carvalho, T. P., Soares, F. A. A. M. N., Vita, R., Francisco, R. P., Basto, J. P., & Alcalá, S. G. (2019). A systematic literature review of machine learning methods applied to predictive maintenance. Computers & Industrial Engineering, 137, 106024.

Fortune Business Insights. (2024). Predictive maintenance market size report. Fortune Business Insights.

Grinberg, M. (2018). Flask web development: Developing web applications with Python (2nd ed.). O'Reilly Media.

Khan, M. A., & Salah, K. (2018). IoT security: Review, blockchain solutions, and open challenges. Future Generation Computer Systems, 82, 395–411.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

OpenAI. (2024). OpenAI API platform documentation. OpenAI.

RAC. (2023). UK vehicle breakdown statistics and common fault analysis. RAC Foundation.

SAE International. (2022). OBD-II standards and vehicle diagnostics. SAE International.

Sharp, H., Rogers, Y., & Preece, J. (2019). Interaction design: Beyond human-computer interaction (5th ed.). Wiley.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Turner, R. (2021). Python and Flask for modern web application development. Packt Publishing.

Van Rossum, G., & Drake, F. L. (2009). Python 3 reference manual. CreateSpace.

Zonta, T., da Costa, C. A., da Rosa Righi, R., de Lima, M. J., da Trindade, E. S., & Li, G. P. (2020). Predictive maintenance in Industry 4.0: A systematic literature review. Computers & Industrial Engineering, 150, 106889.

12. Dissertation declaration

I agree that, should the University wish to retain it for reference purposes, a copy of my dissertation may be held by Bournemouth University normally for a period of three academic years. I understand that once this retention period has expired, my dissertation will be destroyed, except where it has been uploaded to an approved archive, in which case it may be retained beyond this period.

Confidentiality. I confirm that this dissertation does not contain information of a commercial or confidential nature or include personal information other than that which would normally be in the public domain unless the relevant permissions have been obtained. In particular, any information which identifies a particular individual's religious or political beliefs, information relating to their health, ethnicity, criminal history or sex life has been anonymised unless permission has been granted for its publication from the person to whom it relates.

Originality and Use of Generative AI. I declare that all of the work is in accordance with Section 6 of the assignment brief.

Copyright. The copyright for this dissertation remains with me.

Request for Information. I agree that this dissertation may be made available as the result of a request for information under the Freedom of Information Act.

Signed:___Dasheill____

Name: Dasheill Knicx Vas

Date: 14th May 2026

Programme: BSc (Hons) Software Engineering

ORIGINAL WORK DECLARATION

This dissertation and the project that it is based on are my own work, except where stated, in accordance with University regulations.

Signed:_____Dasheill____
.

Name: Dasheill Knicx Vas

Date: 14th May 2026

13. Appendices

- Appendix A – Project Proposal Summary
- Appendix B – Research Ethics Summary
- Appendix C – Software and Development Environment
- Appendix D – Functional Requirements
- Appendix E – Non-Functional Requirements

- Appendix F – System Architecture Summary
- Appendix G – Testing Summary
- Appendix H – Future Improvements

Appendix A

Project Proposal Summary

Project Title

DriveSense AI: An Intelligent Automotive Diagnostic and Predictive Maintenance System Using Machine Learning and OBD-II Data

Project Overview

The aim of this project was to design and develop an intelligent automotive diagnostic platform capable of analysing vehicle data obtained through OBD-II communication and presenting users with real-time diagnostics, predictive maintenance alerts and AI-assisted fault explanations. The system was developed as a web-based application using Python and Flask, with a responsive user interface designed for both desktop and mobile environments.

The project addressed the growing demand for accessible vehicle diagnostics and predictive maintenance technologies. Existing commercial systems often require expensive subscriptions, specialist hardware or advanced technical knowledge. DriveSense AI was developed to provide a more user-friendly and intelligent alternative capable of assisting ordinary vehicle owners as well as automotive enthusiasts.

Original Objectives

Objective ID	Objective Description
OBJ1	Develop a functional automotive diagnostics platform
OBJ2	Integrate OBD-II vehicle communication
OBJ3	Implement live vehicle telemetry monitoring
OBJ4	Develop AI-assisted fault code explanations
OBJ5	Integrate predictive maintenance functionality
OBJ6	Design a responsive and modern user interface
OBJ7	Evaluate system usability and functionality through testing

Proposed Technologies

Technology	Purpose
Python	Backend application development
HTML5	Frontend page structure
Flask Framework	Web application framework
OBD-II Communication	Vehicle telemetry and diagnostics
Machine Learning	Predictive maintenance analysis
AI Diagnostic Interpretation	Intelligent fault explanation

Visual Studio Code	Software development environment
Draw.io	Software for diagrams
Figma	User interface prototyping
CSS3	User interface styling
JavaScript	Interactive frontend functionality

Expected Outcomes

The expected outcome of the project was a fully operational prototype capable of:

- Displaying live vehicle data
- Detecting and interpreting diagnostic trouble codes
- Providing predictive maintenance recommendations
- Delivering an interactive and user-friendly experience

Appendix B

Research Ethics Summary

The project was conducted in accordance with Bournemouth University ethical guidelines. No sensitive personal data was collected during the development of the system. User testing participants voluntarily interacted with the prototype and provided usability feedback anonymously.

The project did not involve medical, financial or highly confidential information. All collected testing feedback was anonymised and used solely for academic evaluation purposes.

Ethical Considerations

Ethical Area	Description
Privacy Protection	User information and vehicle data remained confidential
Data Handling	Vehicle diagnostic information was handled securely
AI Transparency	AI-generated recommendations were presented responsibly
User Safety	Misleading or harmful diagnostic guidance was avoided
Participation	User testing was voluntary and anonymised

No harmful or invasive testing procedures were involved during the completion of the project.

Appendix C

Software and Development Environment

Development Tools

Software / Tool	Purpose
Visual Studio Code	Primary development environment
Python 3	Backend programming language
Flask Framework	Web application framework
Git and GitHub	Version control and project management
Figma	Interface prototyping
HTML5	Frontend structure
CSS3	User interface styling
JavaScript	Interactive frontend behaviour

AI and Diagnostic Components

Component	Purpose
OBD-II Communication Libraries	Vehicle communication and telemetry
AI Diagnostic Interpretation	Intelligent fault explanation
Predictive Maintenance Analysis	Maintenance prediction functionality
Real-Time Telemetry Processing	Live vehicle data monitoring

Hardware Used

Hardware	Purpose
Laptop / PC Development Environment	Software development and testing
ENET Ethernet-to-OBD Interface	Vehicle communication
BMW F20 Vehicle	Real-world validation testing

Appendix D

Functional Requirements

Requirement ID	Requirement Description
FR1	The system shall allow users to connect to a vehicle through OBD-II communication
FR2	The system shall display diagnostic trouble codes in real time
FR3	The platform shall provide AI-generated explanations for detected faults
FR4	The system shall display live telemetry data
FR5	The system shall generate predictive maintenance alerts
FR6	The application shall support responsive desktop and mobile layouts
FR7	The platform shall provide a dashboard for vehicle monitoring
FR8	The system shall store service and diagnostic records

Appendix E

Non-Functional Requirements

Requirement ID	Requirement Description
NFR1	The platform shall provide a responsive and modern user interface
NFR2	The system shall maintain reliable performance during live monitoring
NFR3	The application shall be easy to navigate for non-technical users
NFR4	The platform shall ensure secure handling of user and vehicle data

NFR5	The system shall provide fast loading times and smooth interaction

Appendix F

System Architecture Summary

The DriveSense AI platform follows a modular architecture consisting of:

- Frontend user interface layer
- Flask backend server
- AI diagnostic processing module
- Vehicle telemetry processing module
- Predictive maintenance analysis component
- Database and service management components

The frontend was designed using HTML, CSS and JavaScript, while backend functionality was implemented using Python and Flask. The modular architecture improved maintainability, scalability and future extensibility.

System Components

System Component	Purpose
Frontend Interface	User interaction and dashboard display
Flask Backend	Application routing and backend processing
AI Module	Intelligent fault analysis and recommendations
Telemetry Module	Real-time vehicle data processing
Database Layer	Service history and scan data management
Predictive Maintenance Engine	Maintenance risk prediction

Appendix G

Testing Summary

Functional Testing

Functional testing was conducted to verify:

- Navigation between pages
- Vehicle connection simulation
- Diagnostic scanning functionality
- AI explanation generation
- Live telemetry updates
- Predictive maintenance outputs

Practical Validation Testing

Practical validation testing was conducted using a BMW F20 vehicle, ENET Ethernet-to-OBD communication interface, E-Sys diagnostic software, and real-time telemetry verification procedures.

Testing Results

Test Case	Expected Result	Outcome

Vehicle connection	Successful ECU communication	Pass
Live telemetry monitoring	Real-time data display	Pass
AI fault explanation	Generate repair guidance	Pass
Predictive maintenance	Display maintenance alerts	Pass
Dashboard responsiveness	Responsive interface across devices	Pass
Service record management	Store and display maintenance records	Pass

Test Outcomes

The testing process demonstrated that:

- The system successfully completed major diagnostic functions.
- Users found the interface modern and easy to navigate.
- AI-assisted explanations improved understanding of vehicle faults.
- Minor improvements could still be made to prediction accuracy and advanced analytics.

Appendix H

Future Improvements

Potential future developments include:

Future Improvement	Description
Physical OBD-II Integration	Full integration with production-grade diagnostic hardware
Cloud Storage	Cloud-based vehicle telemetry and diagnostic storage
Advanced Machine Learning	Improved predictive maintenance model training
Mobile Application Deployment	Native mobile application support
Enhanced Predictive Analytics	Advanced maintenance forecasting capabilities
Voice-Controlled AI Assistant	Hands-free automotive diagnostic interaction
Cybersecurity Enhancements	Improved protection for connected vehicle systems
Fleet Telemetry Analytics	Cloud-based multi-vehicle telemetry analysis
Real-Time ECU Analytics	Advanced CAN bus and ECU behaviour monitoring

Appendix I

Extended Figure Descriptions

Figure 1 – DriveSense AI Main Dashboard Interface

Figure 1 illustrates the primary dashboard interface of the DriveSense AI platform. The interface was developed using Python and the Flask web framework, with HTML, CSS and JavaScript used for frontend implementation. The dashboard provides users with access to vehicle diagnostics, live telemetry, predictive maintenance features and AI-assisted analysis within a single integrated environment.

Figure 2 – DriveSense AI Live Vehicle Telemetry Interface

Figure 2 shows the live telemetry interface used to display real-time vehicle sensor data within the DriveSense AI platform. The interface presents key monitoring values including engine RPM, vehicle speed, coolant temperature, engine load, throttle position and fuel level. This supports the diagnostic purpose of the system by allowing users to observe vehicle behaviour during testing.

Figure 3 – DriveSense AI Fault Analysis

Figure 3 demonstrates the diagnostic fault analysis interface within the DriveSense AI platform. The interface displays detected diagnostic trouble codes (DTCs), affected vehicle systems, severity scores and AI-assisted recommendations. The system provides simplified explanations, likely causes and recommended actions to support users in understanding vehicle faults and maintenance requirements.

Figure 4 – DriveSense AI Fault Analysis and AI Recommendation Interface

Figure 4 demonstrates the AI-assisted recommendation interface integrated within the DriveSense AI platform. The interface presents fault descriptions, severity classifications, repair suggestions and driving recommendations generated through AI-assisted analysis. The system was designed to simplify automotive diagnostics by translating technical fault information into understandable maintenance guidance for users.

Figure 5 – DriveSense AI Diagnostics Interface

Figure 5 illustrates the diagnostics interface developed for the DriveSense AI platform. The interface enables users to view active diagnostic trouble codes (DTCs), predictive fault analysis results and AI-assisted recommendations within a unified dashboard environment. The system integrates real-time diagnostic monitoring with machine learning-based vehicle health assessment to support intelligent maintenance decision-making.

Figure 6 – DriveSense AI Predictive Maintenance and AI Analytics Interface

Figure 6 presents the predictive maintenance and AI analytics interface developed for the DriveSense AI platform. The module analyses vehicle telemetry, mileage history, active diagnostic faults and service information in order to estimate vehicle health conditions and identify potential maintenance risks before critical failures occur. Machine learning techniques were incorporated to generate predictive health scores, maintenance urgency indicators and confidence estimates to support proactive automotive maintenance decision-making.

Figure 7 – DriveSense AI System Architecture Diagram

Figure 7 - The system architecture of the DriveSense AI platform was designed using a modular layered structure to improve maintainability, scalability and integration between diagnostic services. The architecture begins with

the DriveSense AI user interface developed using Flask, HTML, CSS and JavaScript. User requests are processed through the Flask backend application server, which coordinates communication between the diagnostics module, AI analysis engine and live telemetry processing components.

The diagnostics module manages OBD-II diagnostic communication and fault code interpretation, while the live telemetry module processes real-time vehicle sensor information including RPM, coolant temperature, throttle position and fuel level data. The AI analysis engine performs predictive maintenance evaluation, vehicle health scoring and intelligent recommendation generation using machine learning-based analysis techniques.

Processed vehicle and user data are stored within the SQLite database environment, including scan history, diagnostic fault logs and service records. The system communicates with the vehicle ECU through the OBD-II interface, enabling real-time telemetry acquisition and automotive diagnostic interaction.

Figure 11 – Extended Explanation

Figure 11 demonstrates the real-world deployment environment used during practical validation of the DriveSense AI platform. The testing configuration included a laptop-based diagnostic environment connected directly to the BMW F20 vehicle through an ENET Ethernet-to-OBD interface. This setup enabled live vehicle communication, telemetry interaction and practical hardware integration testing within a real automotive environment.

Figure 12 – Extended Explanation

Figure 12 demonstrates the ENET Ethernet-to-OBD diagnostic interface used to establish communication between the BMW vehicle and the DriveSense AI testing environment. The interface enabled stable real-time diagnostic communication and telemetry interaction during practical system validation and hardware integration testing.

Figure 13 – Extended Explanation

Figure 13 demonstrates the active Ethernet communication established between the laptop diagnostic environment and the BMW F20 vehicle through the ENET interface. The Windows network configuration confirmed successful real-time data transmission required for ECU communication, telemetry interaction and practical diagnostic testing procedures.

Figure 14 – Extended Explanation

Figure 14 demonstrates the use of BMW E-Sys diagnostic software during practical vehicle communication testing. The interface identified available BMW communication targets and validated successful ENET-based connectivity between the laptop diagnostic environment and the BMW F20 vehicle ECU system during practical testing procedures.

Figure 15 – Extended Explanation

Figure 15 demonstrates the real-world validation environment used to evaluate the DriveSense AI platform during practical BMW F20 diagnostic testing. The testing process involved live interaction with vehicle hardware components, engine bay inspection procedures and AI-assisted diagnostic analysis through the laptop-based DriveSense AI interface. The validation setup confirmed successful integration between software diagnostics, physical vehicle inspection procedures and real-time automotive testing workflows.

Figure 16 – Extended Explanation

Figure 16 demonstrates the operational DriveSense AI dashboard during live BMW F20 vehicle testing. The interface displayed real-time telemetry information including vehicle speed, engine data, drive mode status and tyre pressure monitoring values. Practical validation confirmed successful live data visualisation and real-time communication between the DriveSense AI platform and the vehicle diagnostic environment during telemetry testing procedures.

Figure 17 – Extended Explanation

Figure 17 demonstrates real-time tyre pressure telemetry validation performed between the BMW F20 vehicle and the DriveSense AI telemetry system. The tyre pressure values displayed within the DriveSense AI dashboard matched the live pressure measurements obtained using an external tyre inflator device at 35 PSI. This practical validation confirmed accurate real-time telemetry monitoring and successful synchronisation between the vehicle data system and the DriveSense AI platform.

Figure 18 – Extended Explanation

Figure 18 demonstrates the AI-assisted repair guidance functionality integrated within the DriveSense AI platform. During practical testing, the system recommended the use of a 14 mm socket for the identified maintenance procedure, and the recommendation was validated using a real workshop tool during vehicle inspection activities. This validation confirmed the capability of the AI assistant to provide context-aware repair guidance and practical maintenance support within a live automotive environment.

Figure 19 – Extended Explanation

Figure 19 demonstrates the vehicle service record management functionality implemented within the DriveSense AI platform. The interface displayed maintenance history, service intervals, mileage records and vehicle inspection information obtained during practical BMW F20 testing. Validation confirmed successful integration between the DriveSense AI maintenance management interface and real vehicle service monitoring data.